

Machine Learning

CMPSCI 589

Bruno C. da Silva

bsilva@cs.umass.edu

Introduction to Neural Networks

UMassAmherst
College of Information
& Computer Sciences



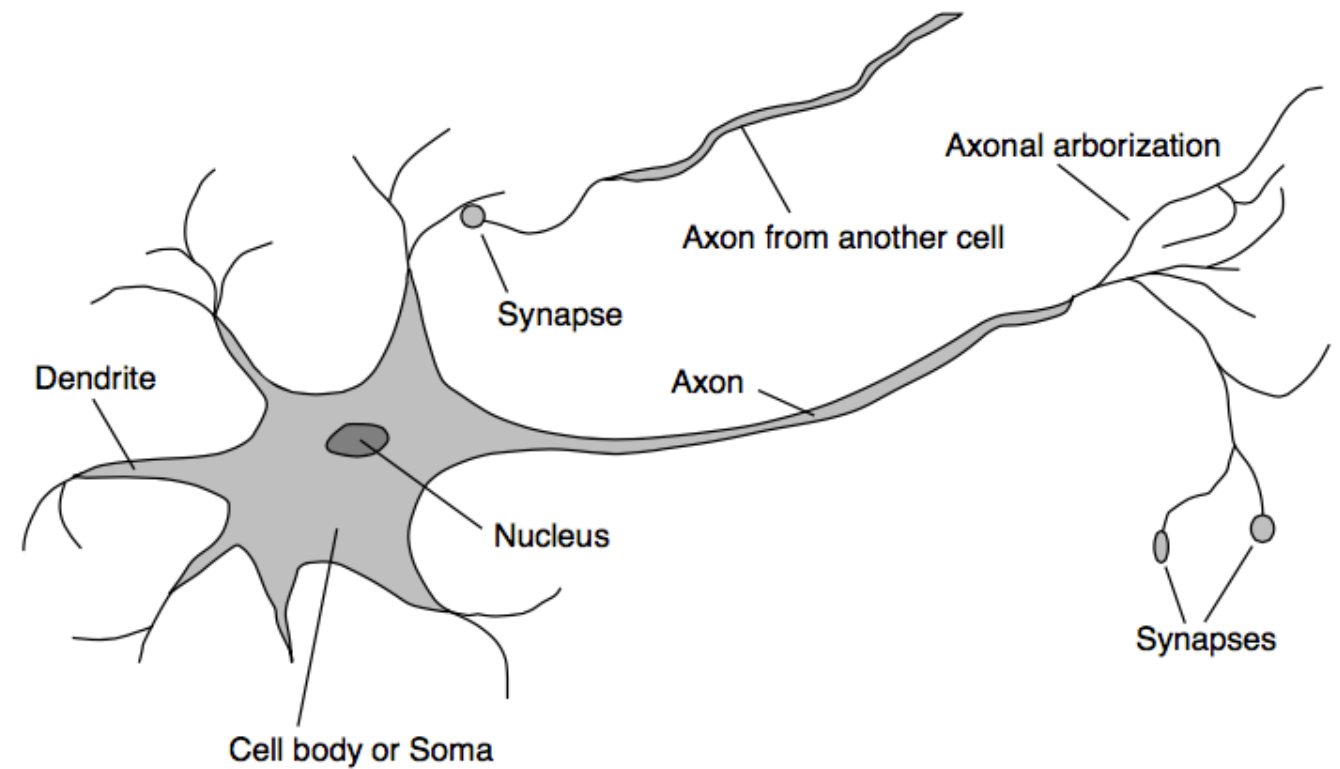
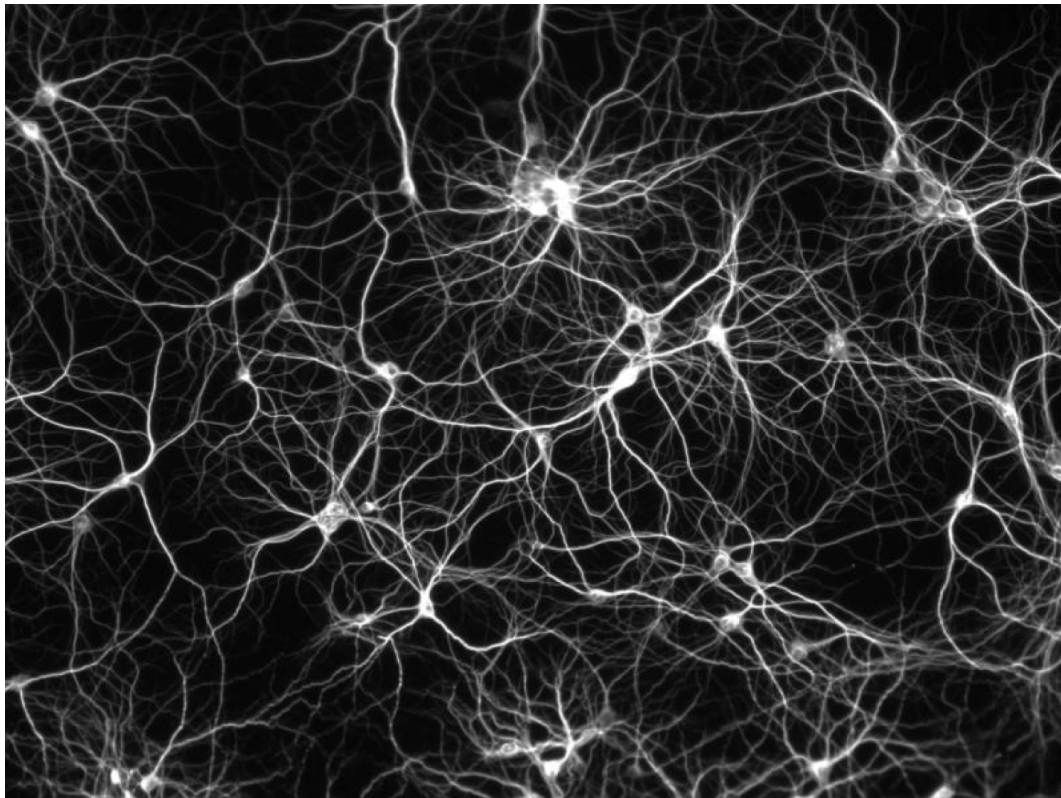
This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/)



Neural Networks

Neural Networks

- **Biologically inspired, originally**
 - (simplified) mathematical model of how biological neurons work
 - network of interconnected neurons → information is propagated/transferred between neurons



Neural Networks

- Pattern recognition
 - image processing, facial recognition, vision for robotics
 - speech recognition
 - large language models
 - machine translation
 - super-human performance in videogames
 - weather forecasting
 - etc.

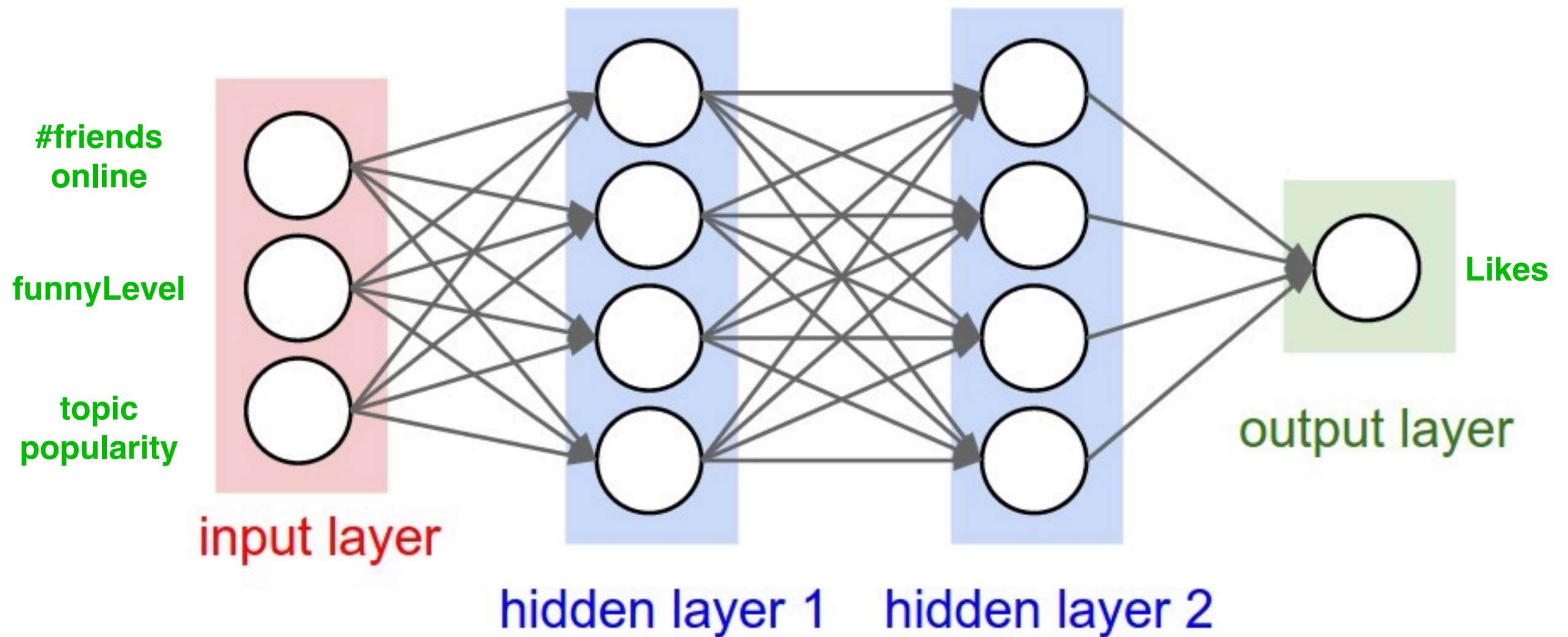
Neural Networks

Tesla Auto-Pilot Saves the Day

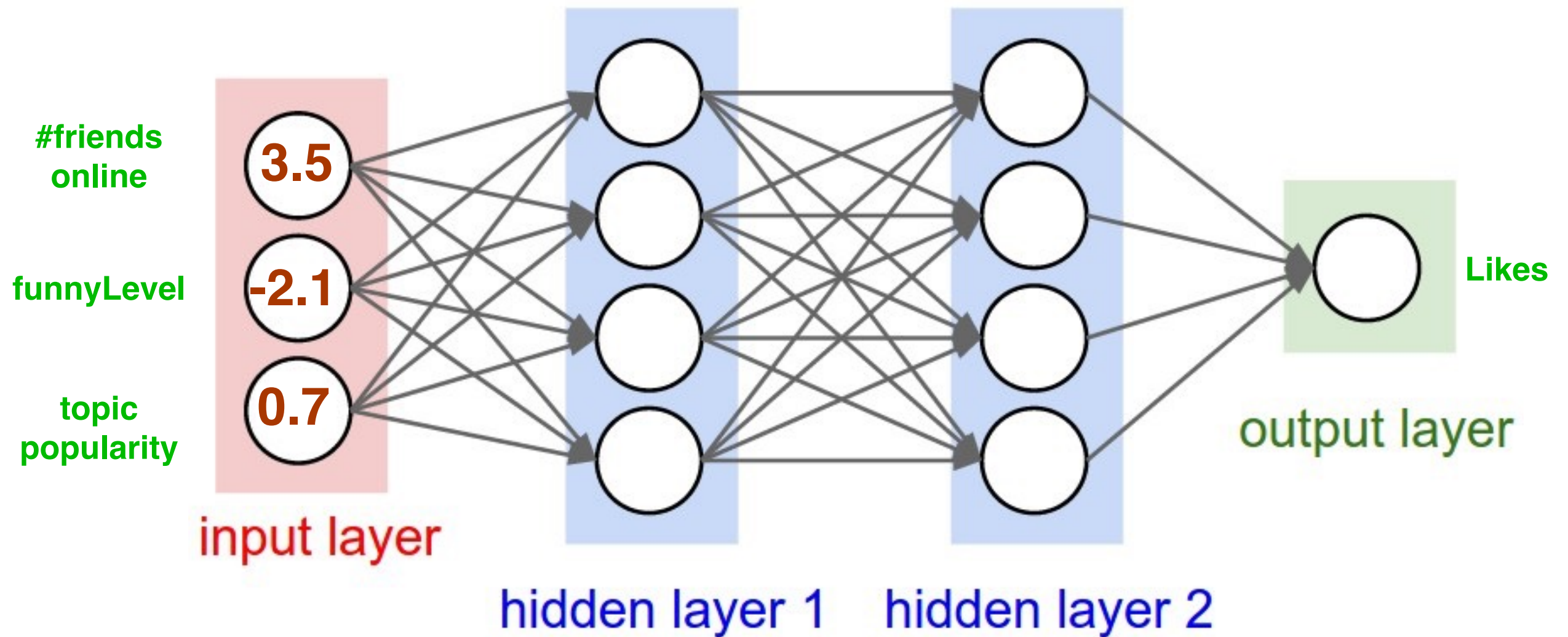


<https://www.youtube.com/watch?v=9X-5fKzmy38>

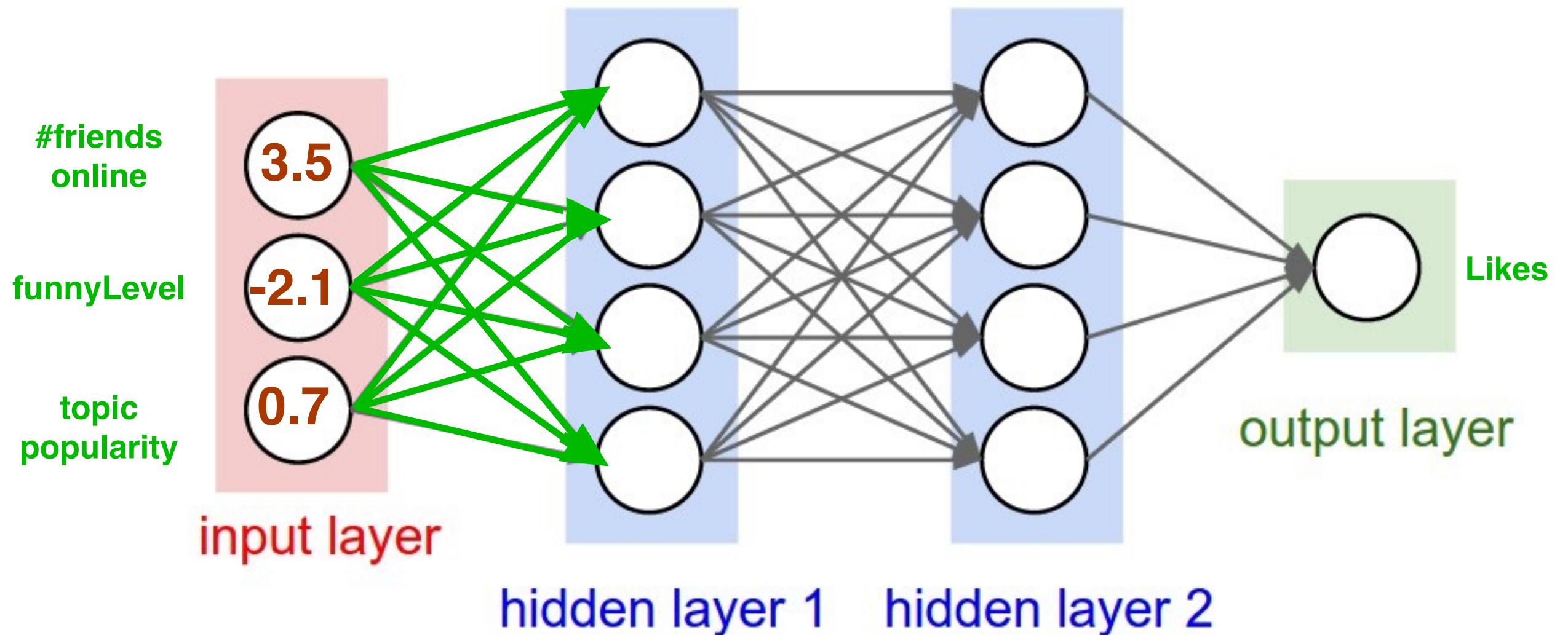
Neural Networks



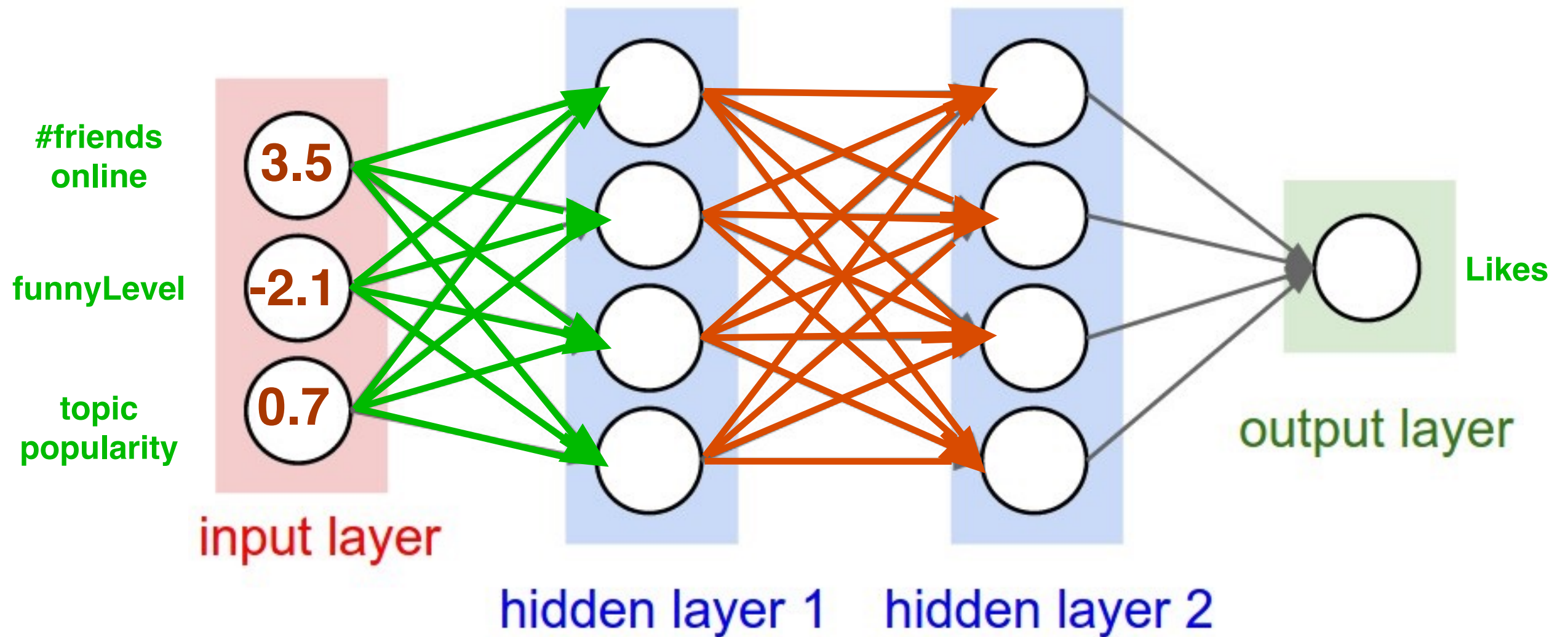
Neural Networks



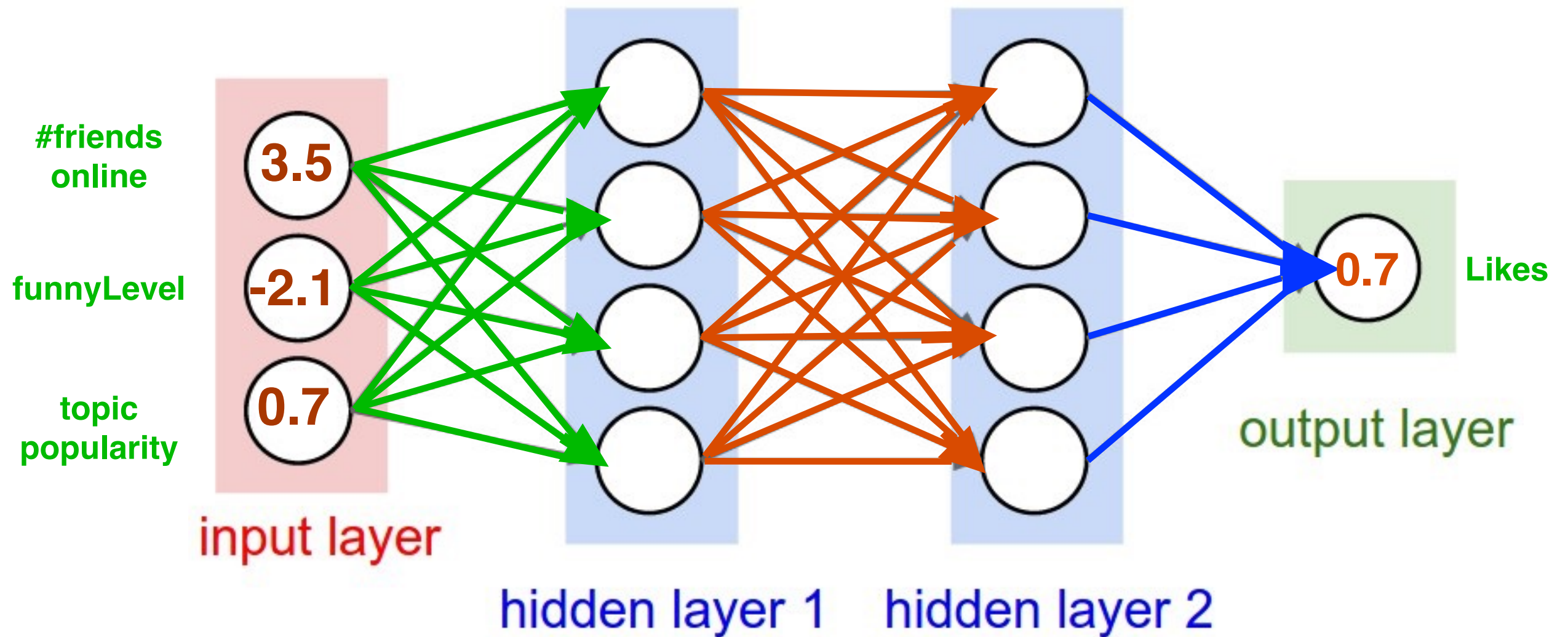
Neural Networks



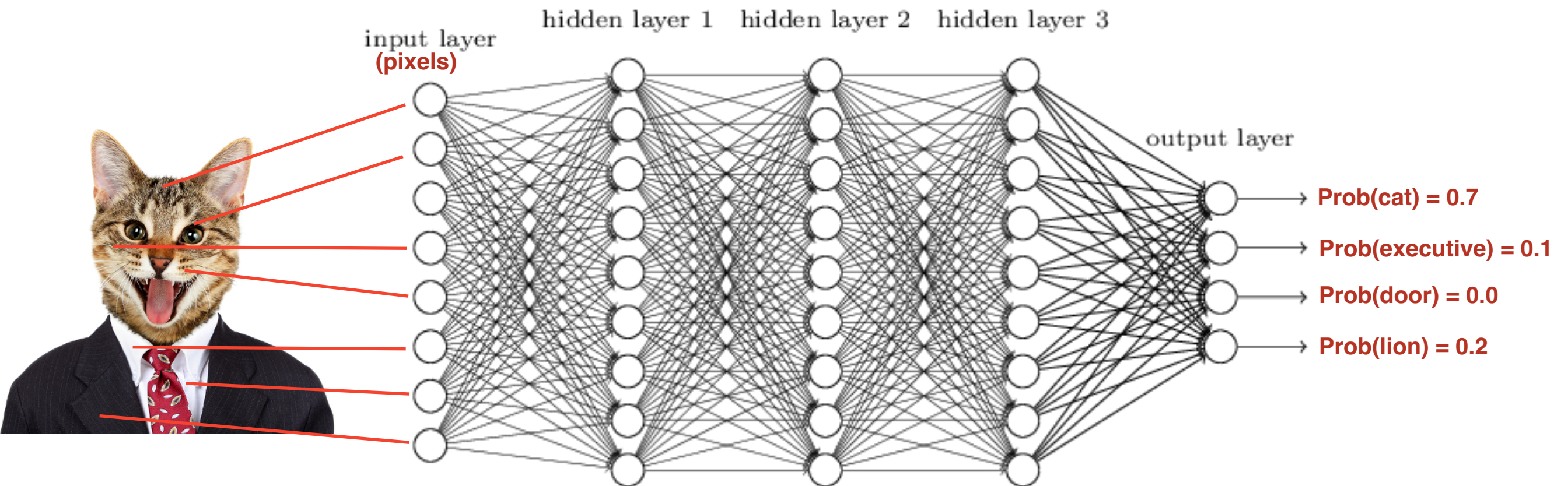
Neural Networks



Neural Networks



Neural Networks

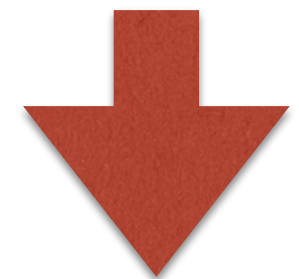


Neural Networks

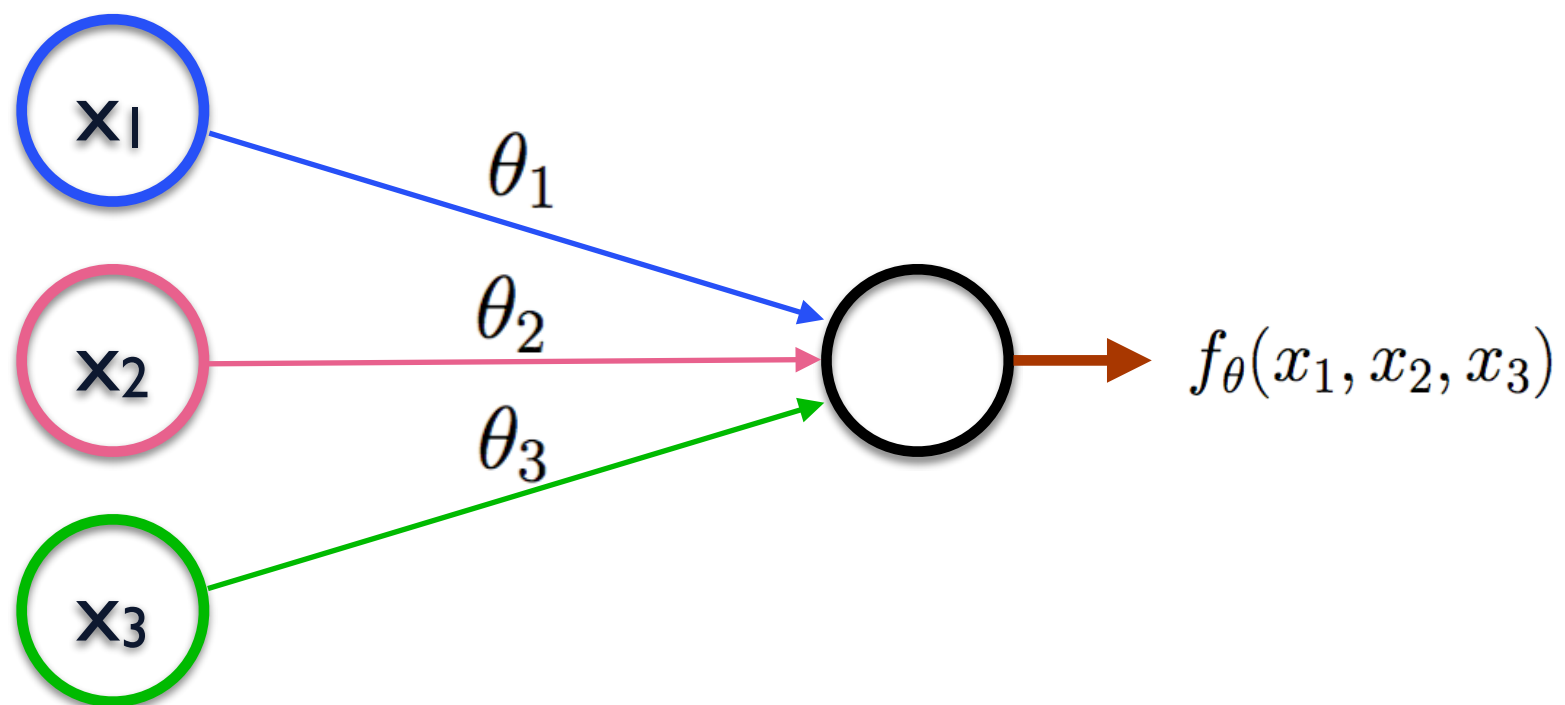
DeepFakes



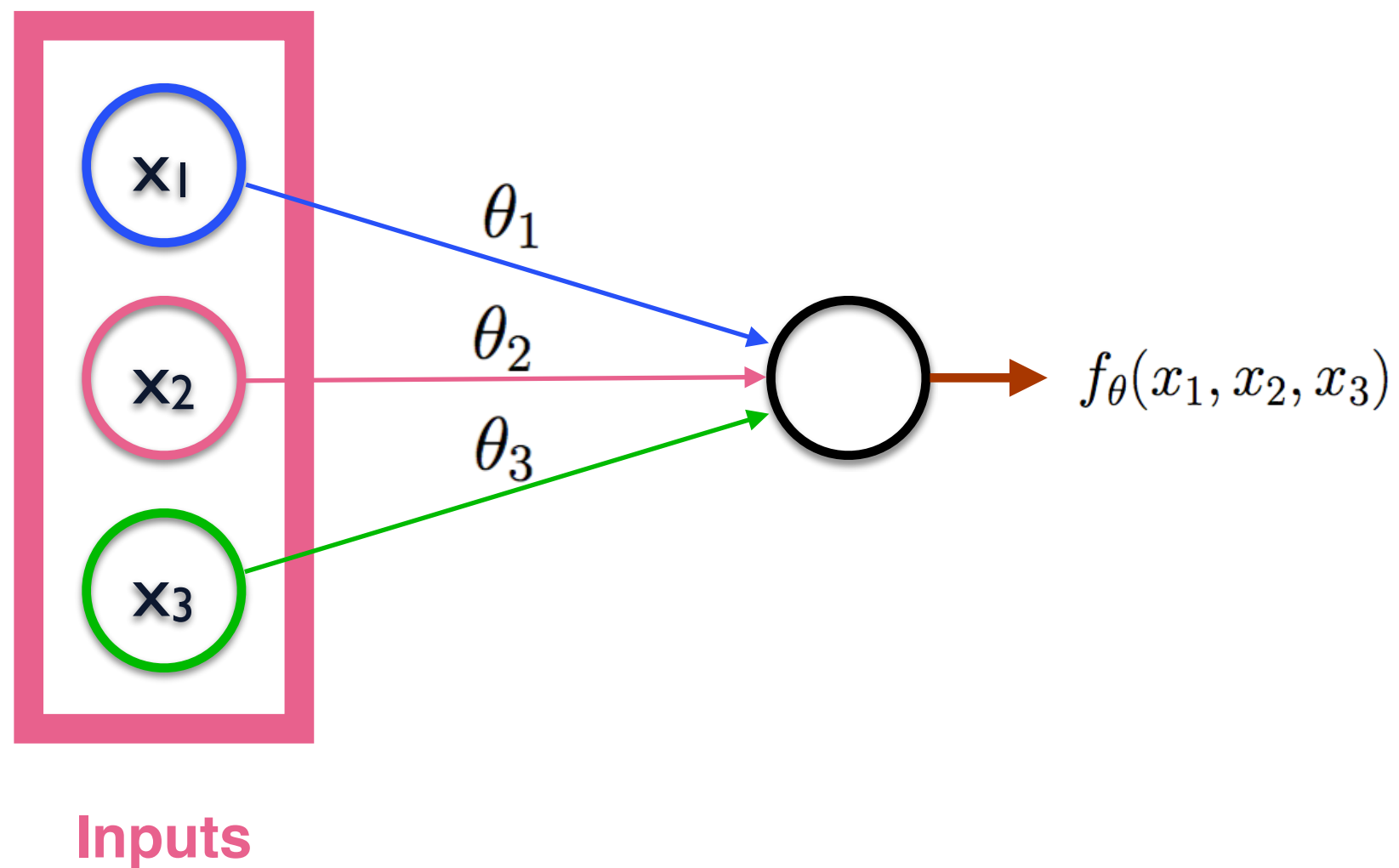
<https://www.youtube.com/watch?v=jl6H-0YWkSc>



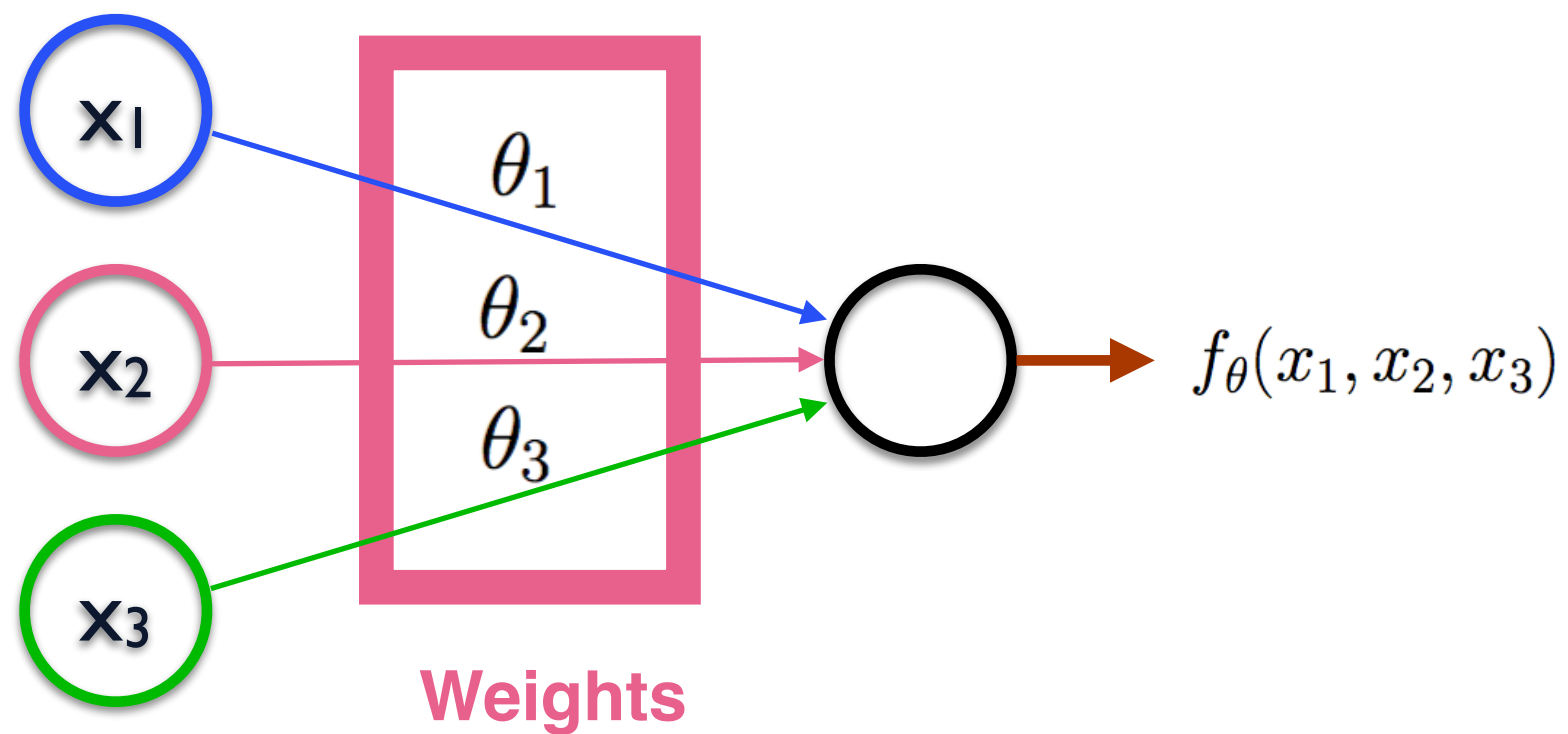
Neuron Model



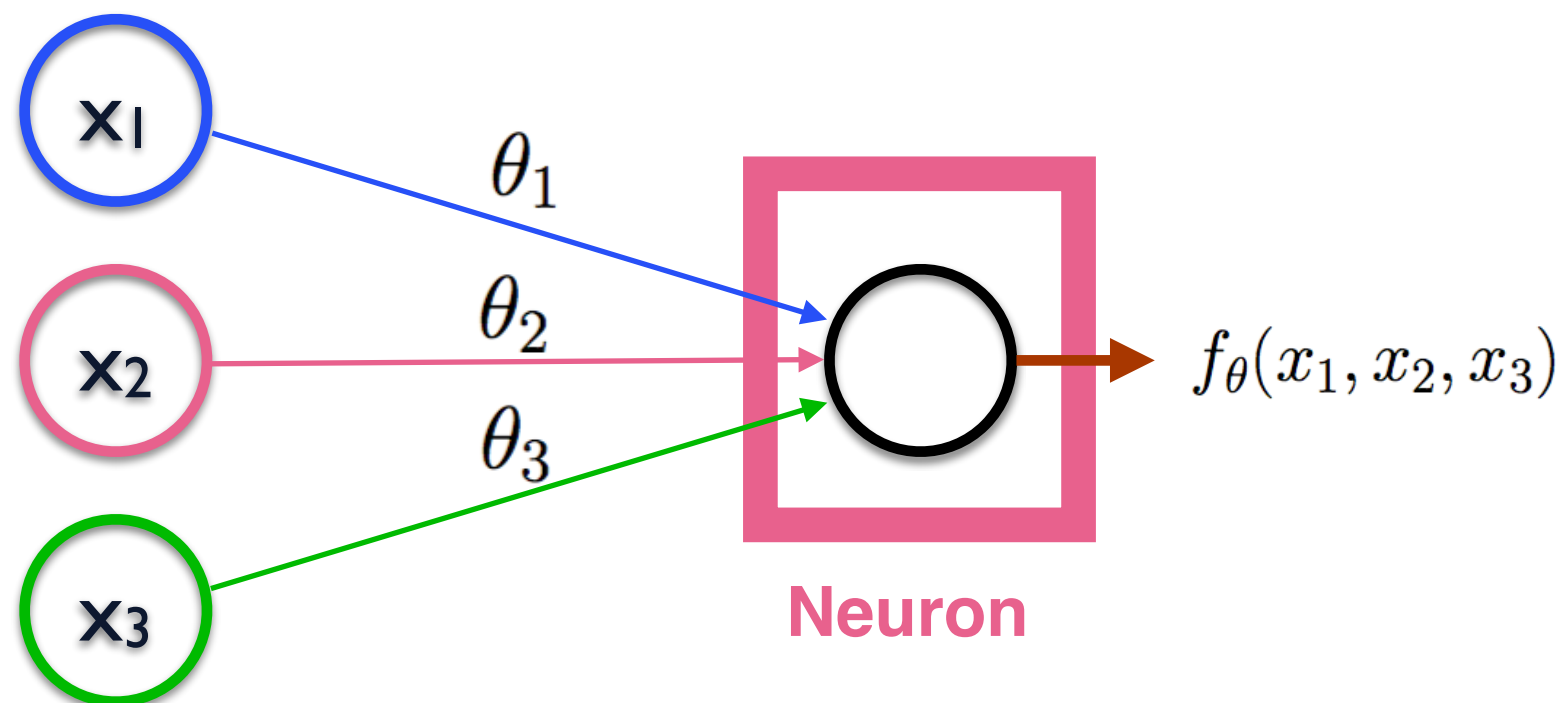
Neuron Model



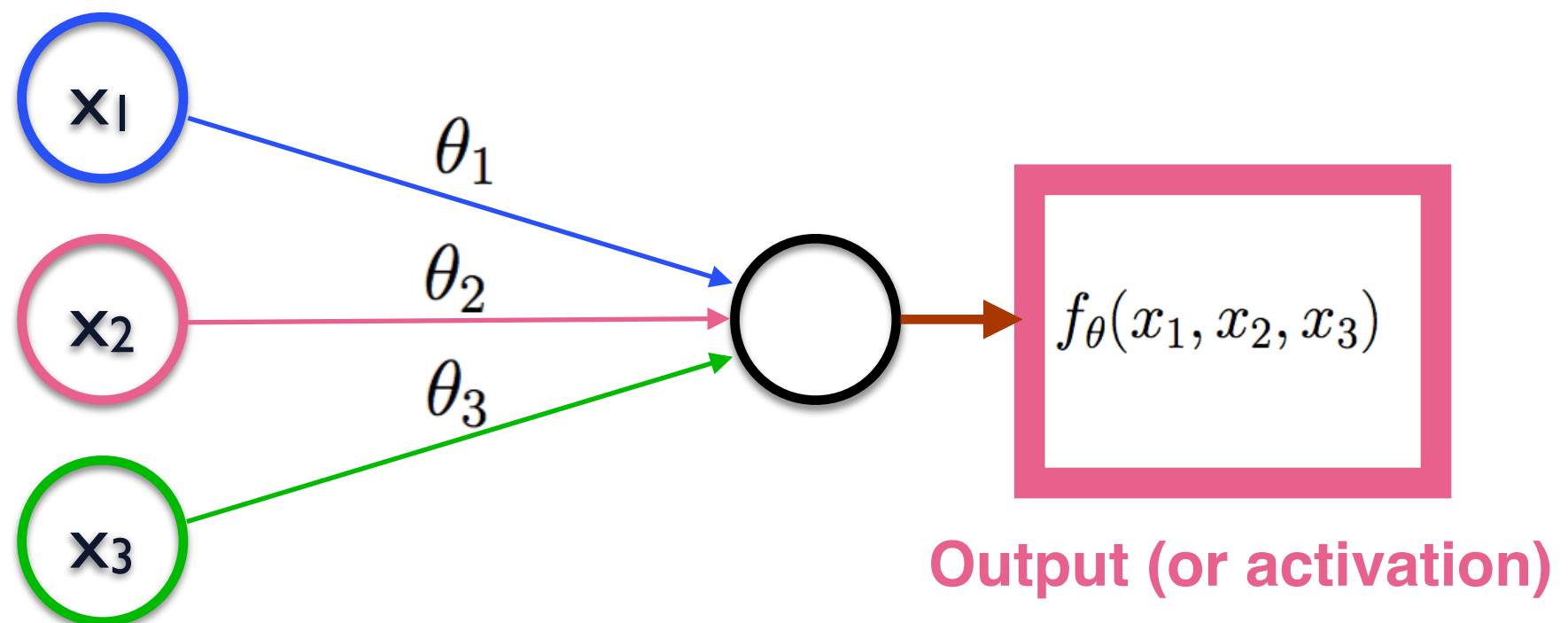
Neuron Model



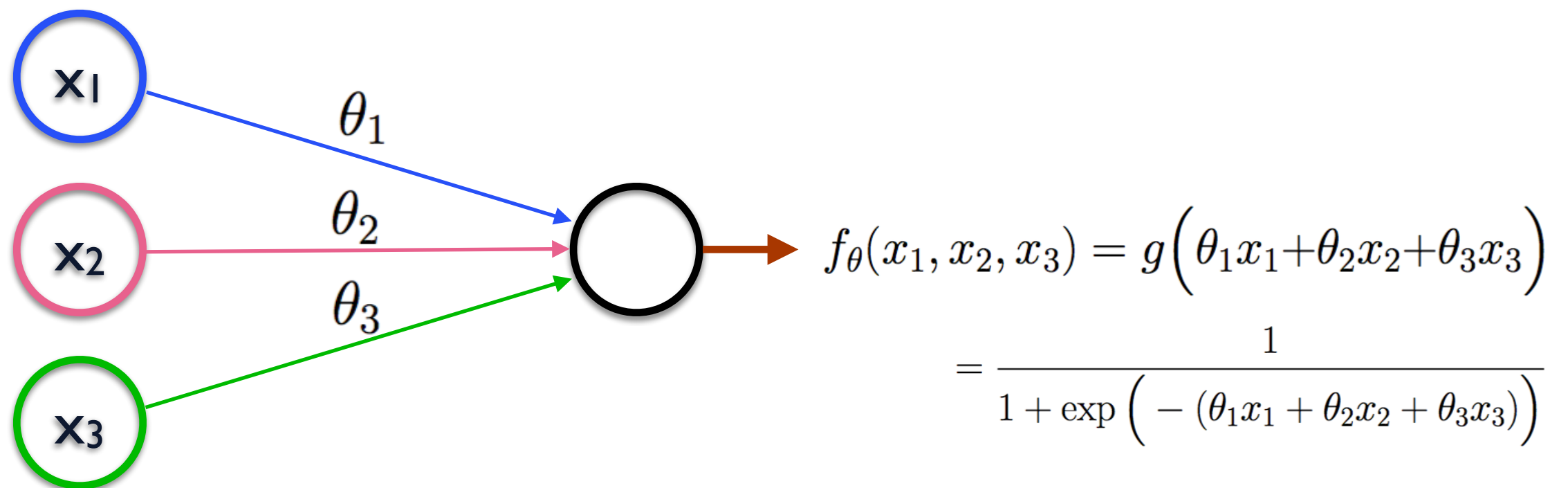
Neuron Model



Neuron Model

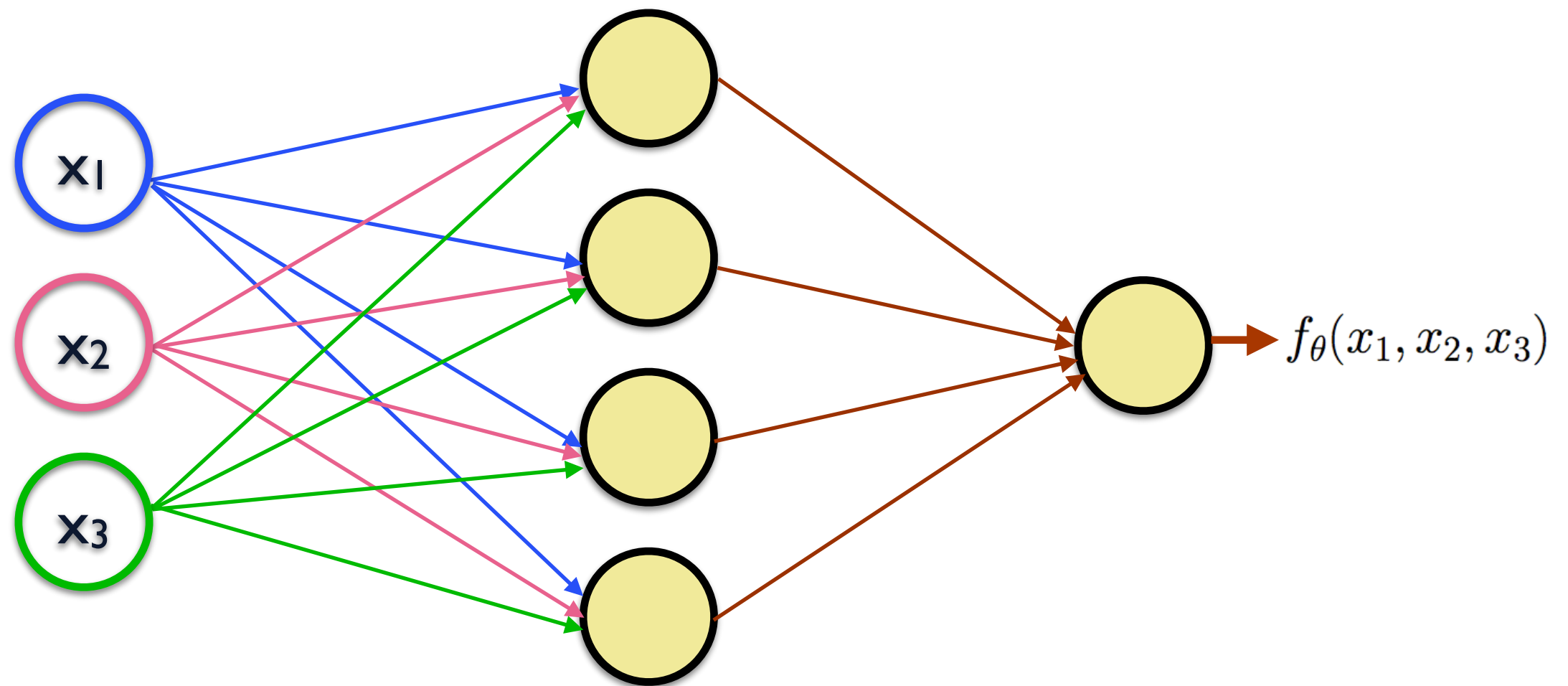


Neuron Model



A single sigmoid neuron = Logistic Regression
Many neurons are combined to form a neural network

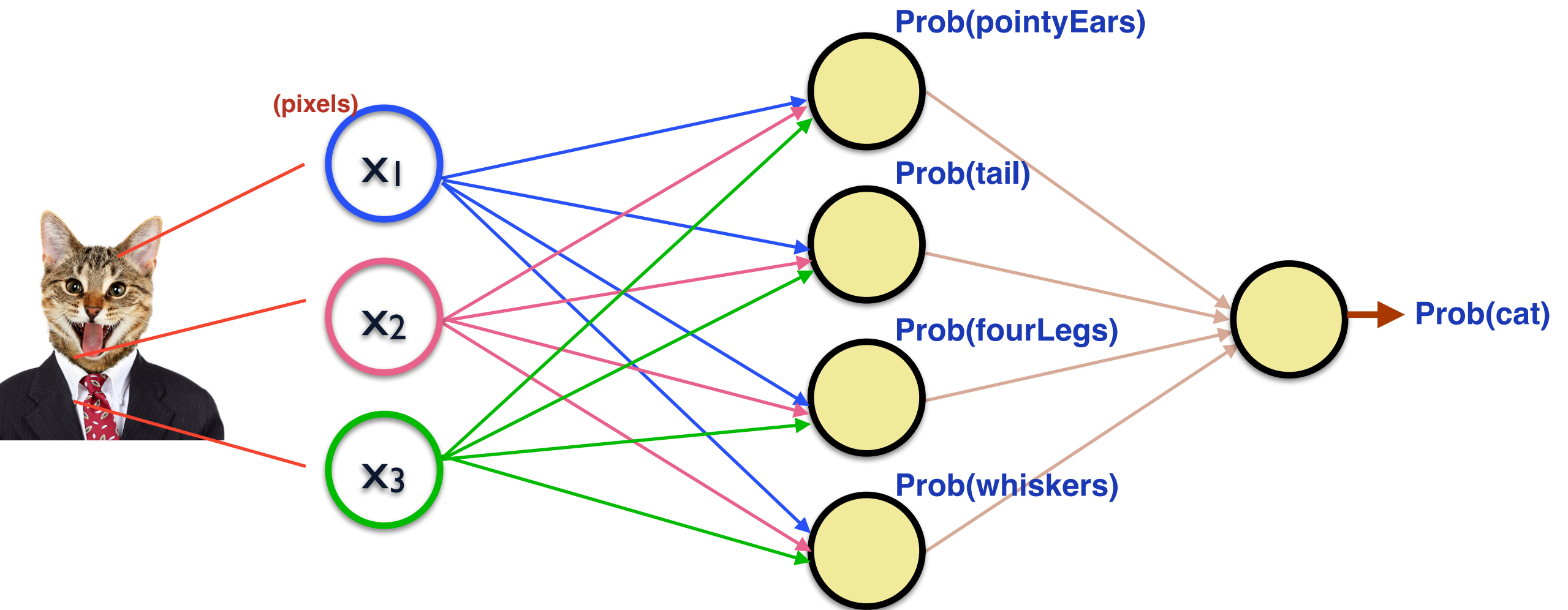
Neural Networks



Network (graph) of connected neurons

Neural Networks

By training the network, hidden neurons automatically learn *features* that are important for the classification task

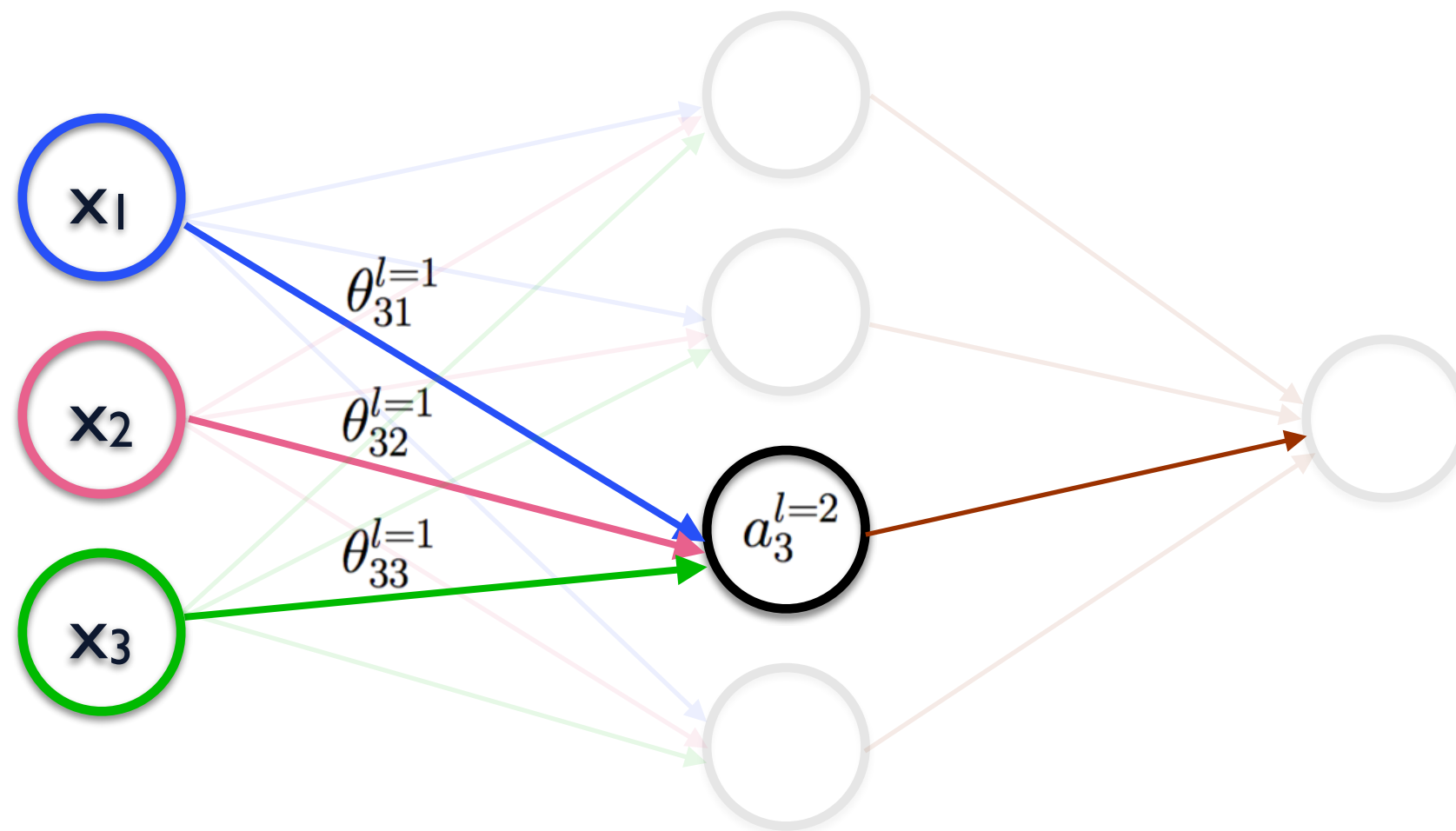


**Why is it advantageous
to combine neurons?**

Neural Networks

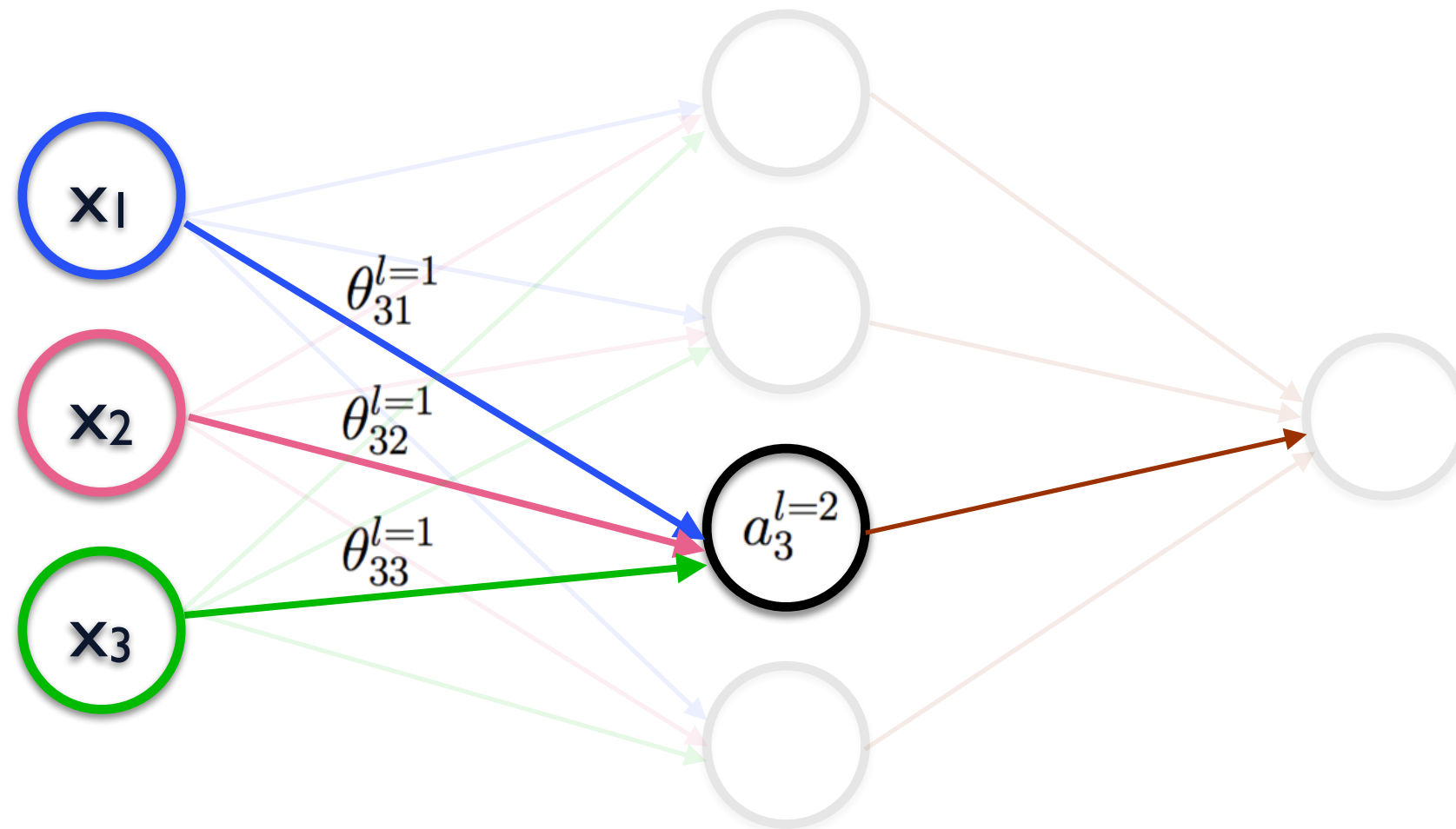
θ_{ij}^l : weight connecting the **j -th** neuron (in the **l -th** layer) to the **i -th** neuron in the next layer

a_j^l : output/activation of the **j -th** neuron in the **l -th** layer



Computing the output/activation of a single neuron

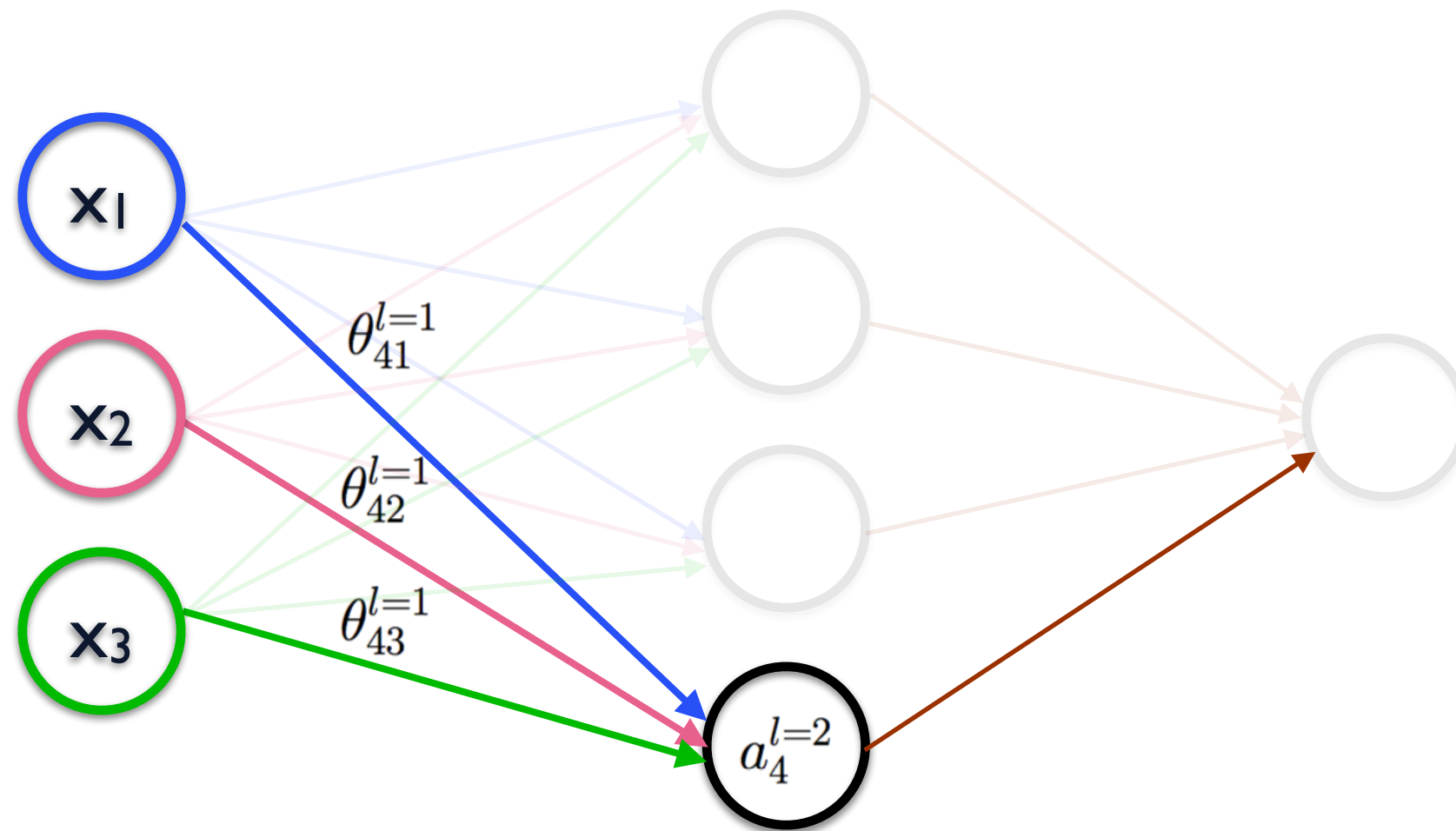
Neural Networks



Computing the output/activation of a single neuron

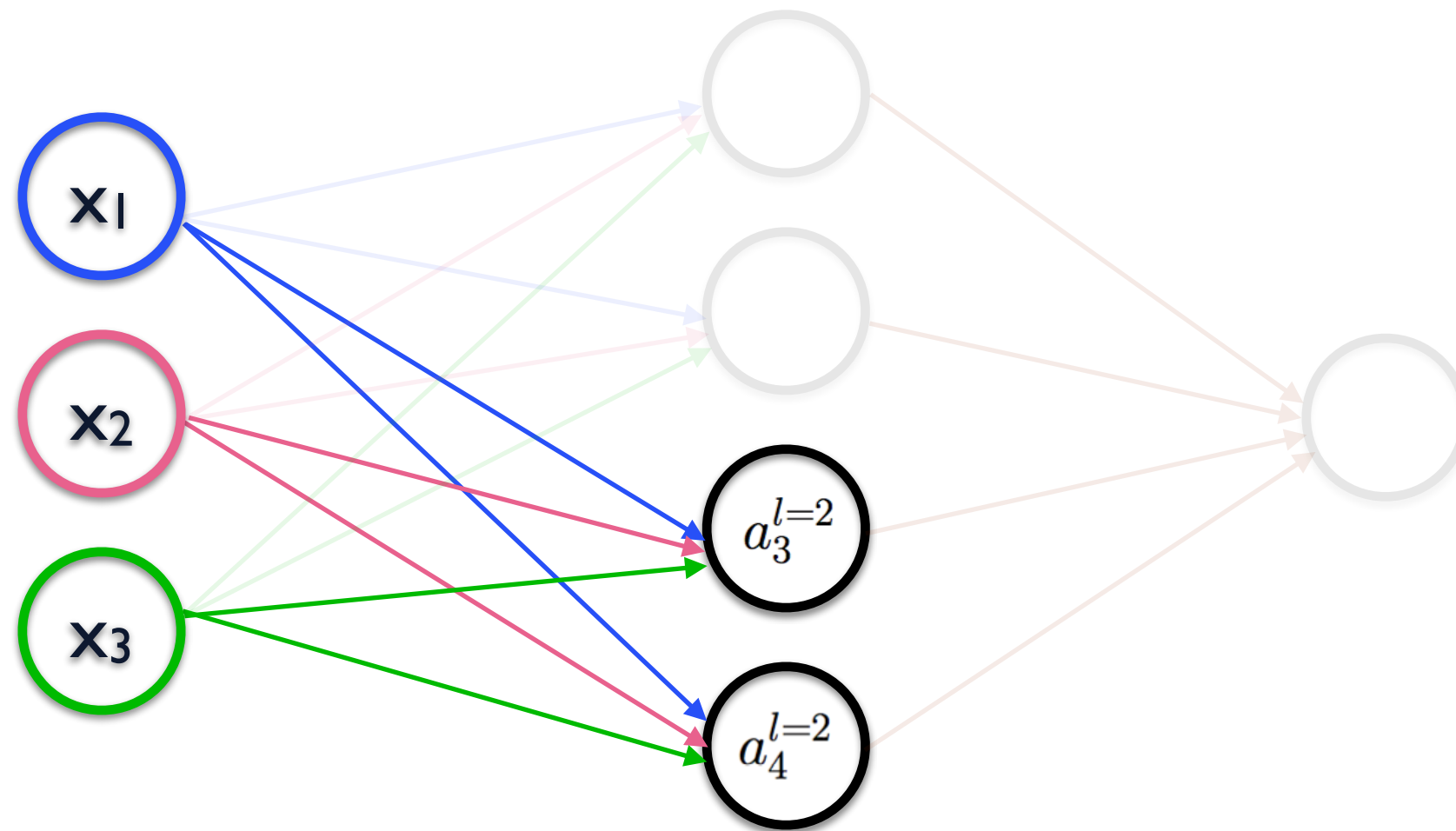
$$a_3^{l=2} = g\left(x_1\theta_{31}^{l=1} + x_2\theta_{32}^{l=1} + x_3\theta_{33}^{l=1}\right)$$

Neural Networks



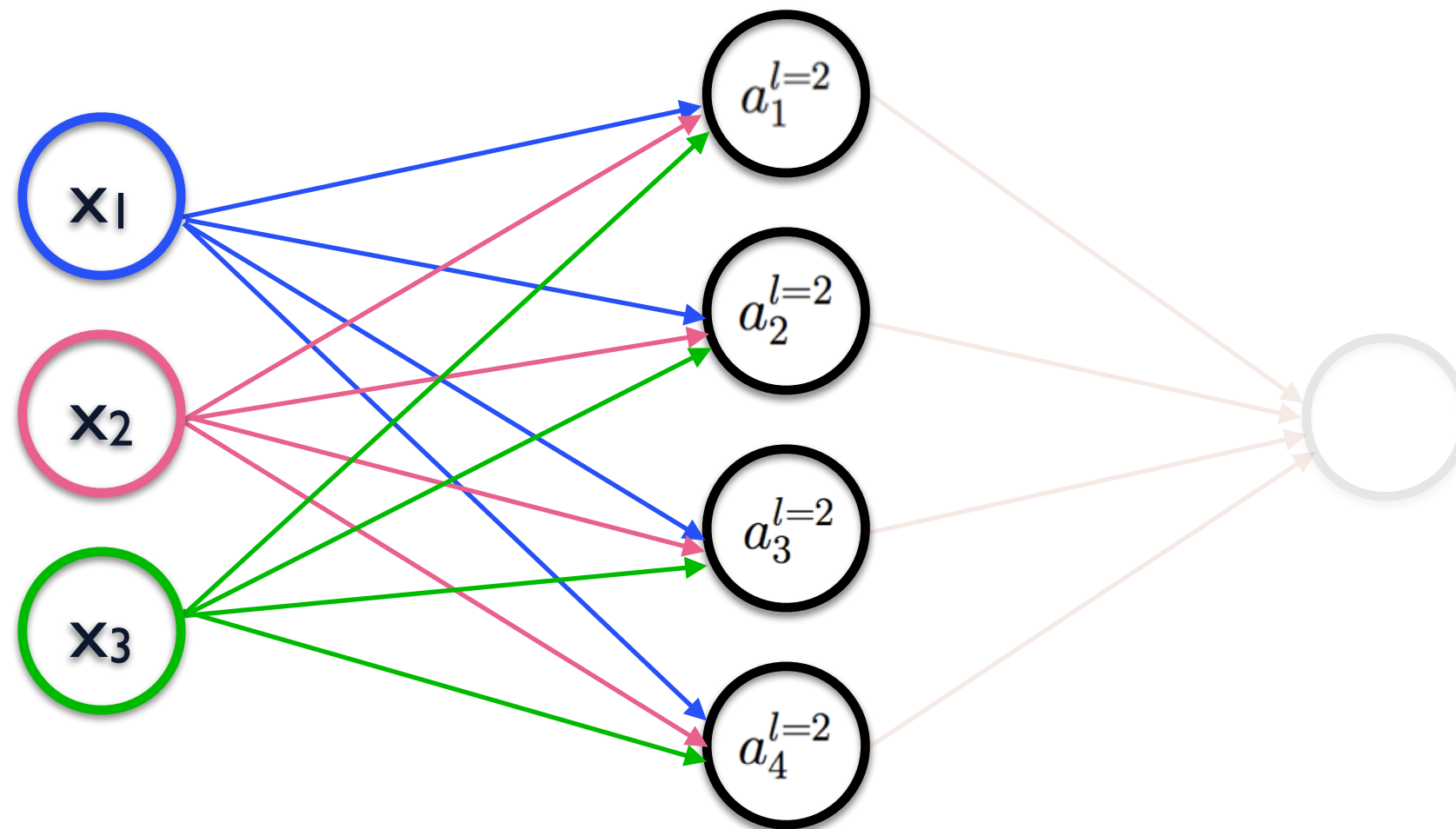
$$a_3^{l=2} = g\left(x_1\theta_{31}^{l=1} + x_2\theta_{32}^{l=1} + x_3\theta_{33}^{l=1}\right)$$
$$a_4^{l=2} = g\left(x_1\theta_{41}^{l=1} + x_2\theta_{42}^{l=1} + x_3\theta_{43}^{l=1}\right)$$

Neural Networks



$$a_3^{l=2} = g\left(x_1\theta_{31}^{l=1} + x_2\theta_{32}^{l=1} + x_3\theta_{33}^{l=1}\right)$$
$$a_4^{l=2} = g\left(x_1\theta_{41}^{l=1} + x_2\theta_{42}^{l=1} + x_3\theta_{43}^{l=1}\right)$$

Neural Networks



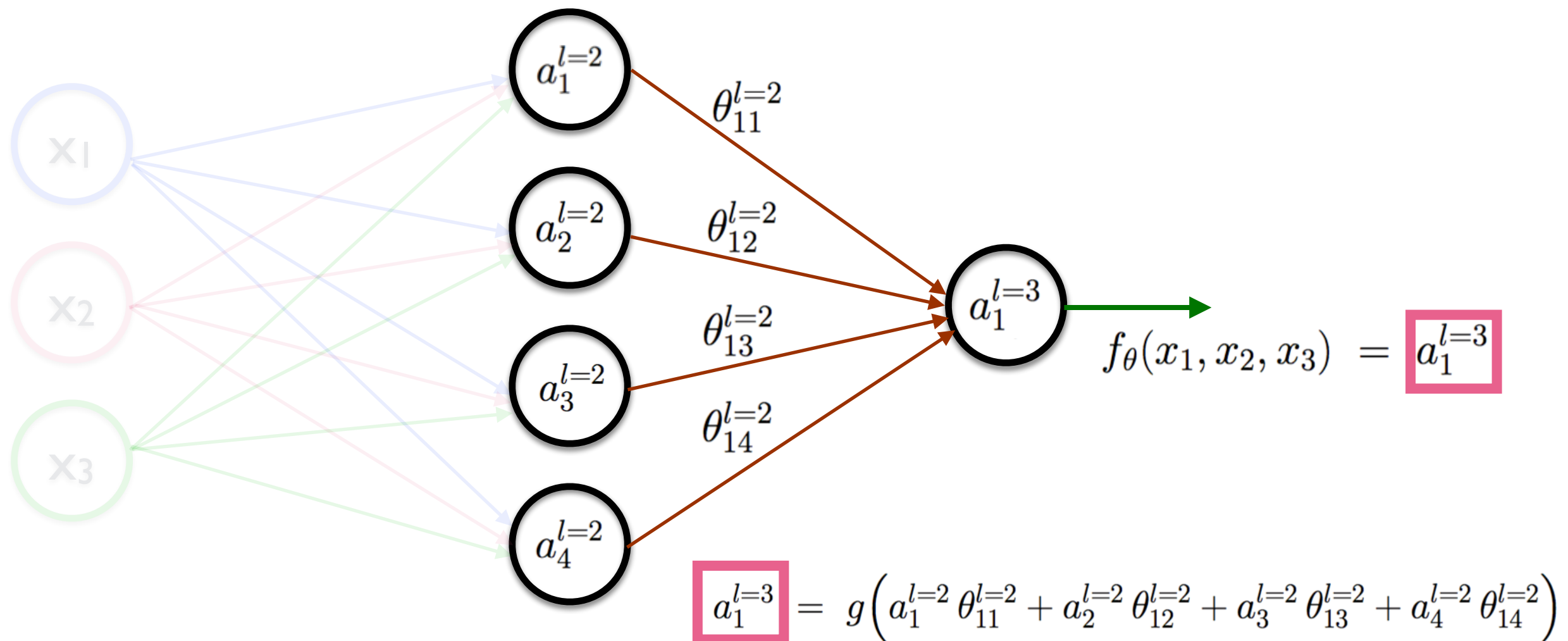
$$a_1^{l=2} = g\left(x_1\theta_{11}^{l=1} + x_2\theta_{12}^{l=1} + x_3\theta_{13}^{l=1}\right)$$

$$a_2^{l=2} = g\left(x_1\theta_{21}^{l=1} + x_2\theta_{22}^{l=1} + x_3\theta_{23}^{l=1}\right)$$

$$a_3^{l=2} = g\left(x_1\theta_{31}^{l=1} + x_2\theta_{32}^{l=1} + x_3\theta_{33}^{l=1}\right)$$

$$a_4^{l=2} = g\left(x_1\theta_{41}^{l=1} + x_2\theta_{42}^{l=1} + x_3\theta_{43}^{l=1}\right)$$

Neural Networks

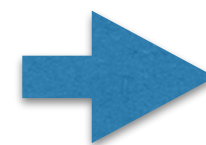


$$a_1^{l=2} = g\left(x_1 \theta_{11}^{l=1} + x_2 \theta_{12}^{l=1} + x_3 \theta_{13}^{l=1}\right)$$

$$a_2^{l=2} = g\left(x_1 \theta_{21}^{l=1} + x_2 \theta_{22}^{l=1} + x_3 \theta_{23}^{l=1}\right)$$

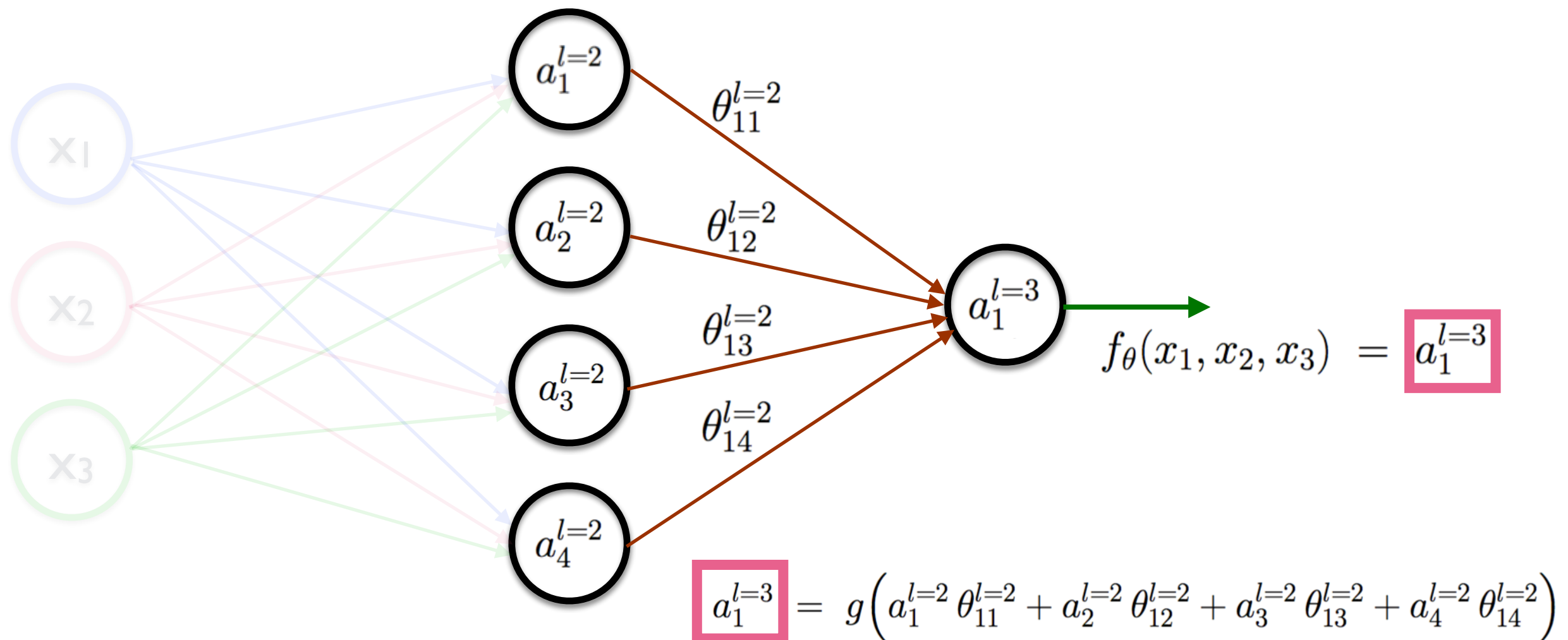
$$a_3^{l=2} = g\left(x_1 \theta_{31}^{l=1} + x_2 \theta_{32}^{l=1} + x_3 \theta_{33}^{l=1}\right)$$

$$a_4^{l=2} = g\left(x_1 \theta_{41}^{l=1} + x_2 \theta_{42}^{l=1} + x_3 \theta_{43}^{l=1}\right)$$



Output/prediction made by the neural network

Neural Networks



$$a_1^{l=2} = g\left(x_1 \theta_{11}^{l=1} + x_2 \theta_{12}^{l=1} + x_3 \theta_{13}^{l=1}\right)$$

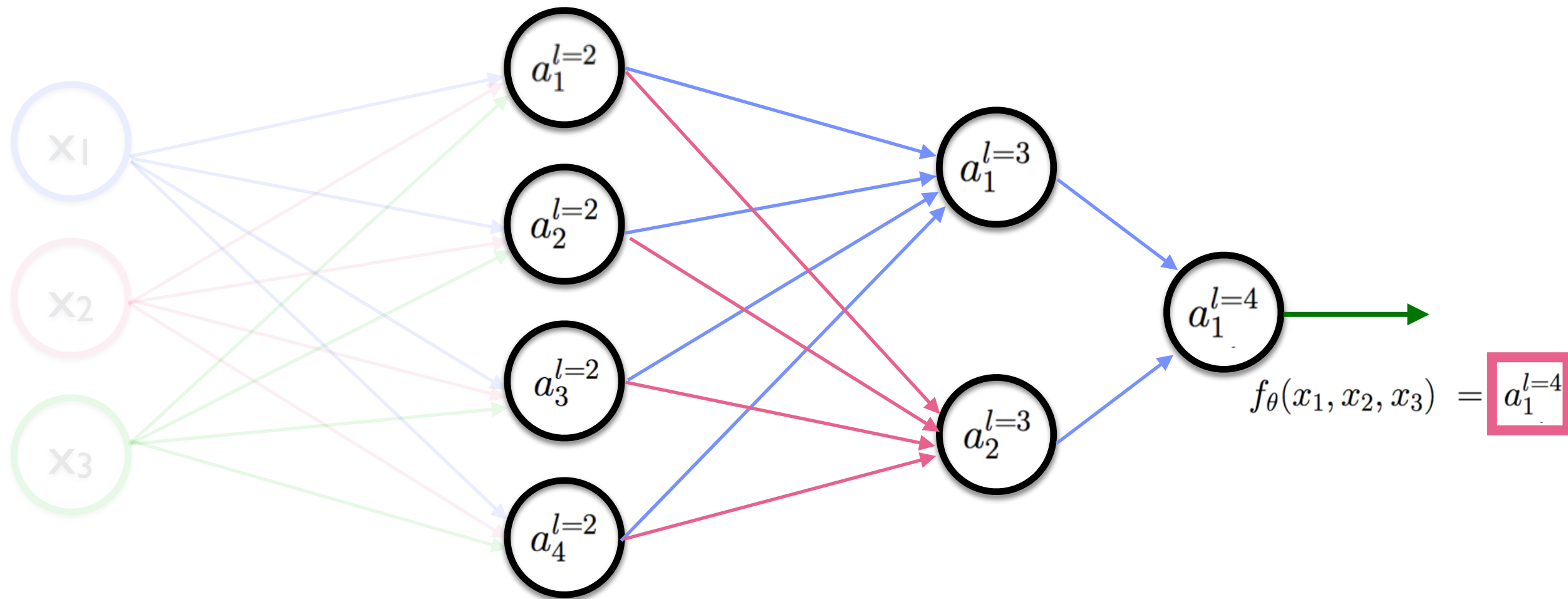
$$a_2^{l=2} = g\left(x_1 \theta_{21}^{l=1} + x_2 \theta_{22}^{l=1} + x_3 \theta_{23}^{l=1}\right)$$

$$a_3^{l=2} = g\left(x_1 \theta_{31}^{l=1} + x_2 \theta_{32}^{l=1} + x_3 \theta_{33}^{l=1}\right)$$

$$a_4^{l=2} = g\left(x_1 \theta_{41}^{l=1} + x_2 \theta_{42}^{l=1} + x_3 \theta_{43}^{l=1}\right)$$

We can create neural networks with even more layers

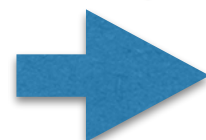
Neural Networks



$$\begin{aligned} a_1^{l=2} &= g\left(x_1\theta_{11}^{l=1} + x_2\theta_{12}^{l=1} + x_3\theta_{13}^{l=1}\right) \\ a_2^{l=2} &= g\left(x_1\theta_{21}^{l=1} + x_2\theta_{22}^{l=1} + x_3\theta_{23}^{l=1}\right) \\ a_3^{l=2} &= g\left(x_1\theta_{31}^{l=1} + x_2\theta_{32}^{l=1} + x_3\theta_{33}^{l=1}\right) \\ a_4^{l=2} &= g\left(x_1\theta_{41}^{l=1} + x_2\theta_{42}^{l=1} + x_3\theta_{43}^{l=1}\right) \end{aligned}$$

$$\begin{aligned} a_1^{l=3} &= g\left(a_1^{l=2}\theta_{11}^{l=2} + a_2^{l=2}\theta_{12}^{l=2} + a_3^{l=2}\theta_{13}^{l=2} + a_4^{l=2}\theta_{14}^{l=2}\right) \\ a_2^{l=3} &= g\left(a_1^{l=2}\theta_{21}^{l=2} + a_2^{l=2}\theta_{22}^{l=2} + a_3^{l=2}\theta_{23}^{l=2} + a_4^{l=2}\theta_{24}^{l=2}\right) \end{aligned}$$

$$\boxed{a_1^{l=4}} = g\left(a_1^{l=3}\theta_{11}^{l=3} + a_2^{l=3}\theta_{12}^{l=3}\right)$$



Network's output/prediction

Neural Networks

Cost Function (J)

Review:

Logistic Regression - Cost Function

$$J(\theta_1, \dots, \theta_K) = \frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1-y^{(i)}) \left(\log(1-f_{\theta}(x^{(i)})) \right)$$

where $f_{\theta}(x_1, \dots, x_K) = g(\theta_1 x_1 + \dots, \theta_K x_K) = \frac{1}{1 + \exp(-(\theta_1 x_1 + \dots, \theta_K x_K))}$ is our hypothesis

We wish to find values for $\theta_0 \dots \theta_K$ that minimize such a cost function, J

Gradient Descent Algorithm

$$\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1, \theta_2) \quad \theta_0 := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1, \theta_2)$$

$$\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1, \theta_2) \quad \theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1, \theta_2)$$

$$\frac{\partial}{\partial \theta_2} J(\theta_0, \theta_1, \theta_2) \quad \theta_2 := \theta_2 - \alpha \frac{\partial}{\partial \theta_2} J(\theta_0, \theta_1, \theta_2)$$

Neural Networks - Cost Function

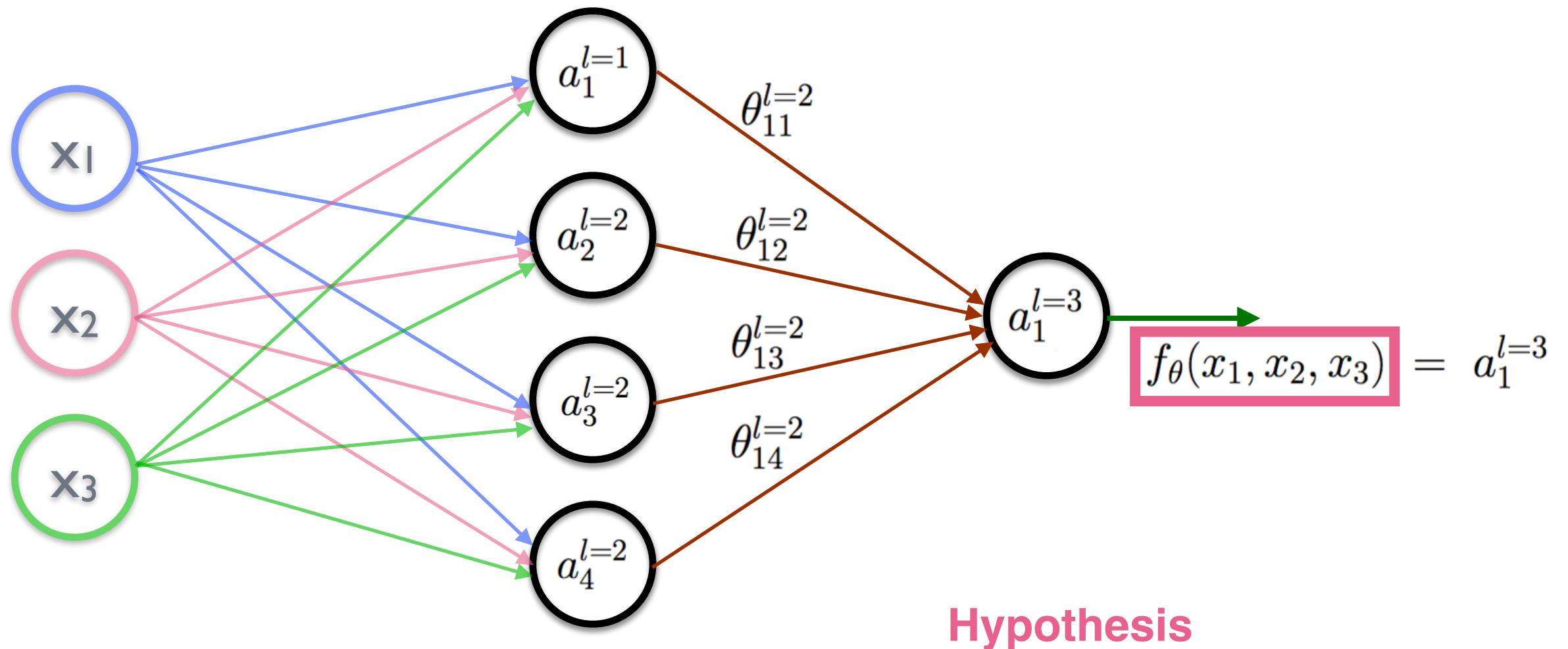
$$J(\theta_1, \dots, \theta_K) = \frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1-y^{(i)}) \left(\log(1-f_{\theta}(x^{(i)})) \right)$$

where $f_{\theta}(x_1, \dots, x_K) = g\left(\theta_1 x_1 + \dots, \theta_K x_K\right) = \frac{1}{1 + \exp\left(-(\theta_1 x_1 + \dots, \theta_K x_K)\right)}$ is our hypothesis

We wish to find values for $\theta_0 \dots \theta_K$ that minimize such a cost function, J

Gradient Descent Algorithm

Neural Networks - Hypothesis



$$f_{\theta}(x_1, x_2, x_3) = a_1^{l=3} = g\left(a_1^{l=2} \theta_{11}^{l=2} + a_2^{l=2} \theta_{12}^{l=2} + a_3^{l=2} \theta_{13}^{l=2} + a_4^{l=2} \theta_{14}^{l=2}\right)$$

where {

$$\begin{aligned} a_1^{l=2} &= g\left(x_1 \theta_{11}^{l=1} + x_2 \theta_{12}^{l=1} + x_3 \theta_{13}^{l=1}\right) \\ a_2^{l=2} &= g\left(x_1 \theta_{21}^{l=1} + x_2 \theta_{22}^{l=1} + x_3 \theta_{23}^{l=1}\right) \\ a_3^{l=2} &= g\left(x_1 \theta_{31}^{l=1} + x_2 \theta_{32}^{l=1} + x_3 \theta_{33}^{l=1}\right) \\ a_4^{l=2} &= g\left(x_1 \theta_{41}^{l=1} + x_2 \theta_{42}^{l=1} + x_3 \theta_{43}^{l=1}\right) \end{aligned}$$

Neural Networks - Cost Function

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \dots, \theta_{11}^{l=L}, \dots) = \frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1-y^{(i)}) \left(\log(1-f_{\theta}(x^{(i)})) \right)$$

where

$$f_{\theta}(x_1, \dots, x_K) = a^{l=L} = \text{function that computes the network's output (output of the last neuron, in the } L\text{-th layer)}$$

is our hypothesis

We wish to find values for each weight θ that minimize such a cost function, J

Gradient Descent Algorithm

$$\begin{array}{ll} \frac{\partial J}{\partial \theta_{11}^{l=1}} & \theta_{11}^{l=1} := \theta_{11}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=1}} \\ \frac{\partial J}{\partial \theta_{12}^{l=1}} & \theta_{12}^{l=1} := \theta_{12}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{12}^{l=1}} \\ \vdots & \vdots \\ \frac{\partial J}{\partial \theta_{11}^{l=2}} & \theta_{11}^{l=2} := \theta_{11}^{l=2} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=2}} \\ \vdots & \vdots \\ \frac{\partial J}{\partial \theta_{11}^{l=L}} & \theta_{11}^{l=L} := \theta_{11}^{l=L} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=L}} \end{array}$$

Neural Networks - Cost Function

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \dots, \theta_{11}^{l=L}, \dots) = \frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1-y^{(i)}) \left(\log(1-f_{\theta}(x^{(i)})) \right)$$

where

$$f_{\theta}(x_1, \dots, x_K) = a^{l=L} = \text{function that computes the network's output (output of the last neuron, in the } L\text{-th layer)}$$

is our hypothesis

We wish to find values for each weight θ that minimize such a cost function, J

Gradient Descent Algorithm

$$\frac{\partial J}{\partial \theta_{11}^{l=1}} \quad \theta_{11}^{l=1} := \theta_{11}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=1}}$$

$$\frac{\partial J}{\partial \theta_{12}^{l=1}} \quad \theta_{12}^{l=1} := \theta_{12}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{12}^{l=1}}$$

⋮

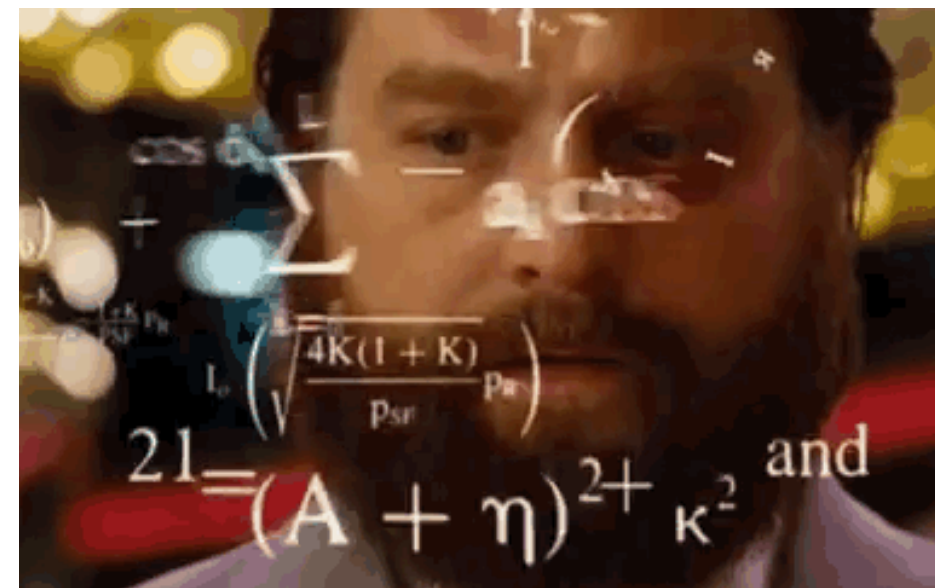
⋮

$$\frac{\partial J}{\partial \theta_{11}^{l=2}} \quad \theta_{11}^{l=2} := \theta_{11}^{l=2} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=2}}$$

⋮

⋮

$$\frac{\partial J}{\partial \theta_{11}^{l=L}} \quad \theta_{11}^{l=L} := \theta_{11}^{l=L} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=L}}$$



How to compute these gradients?

Neural Networks - Cost Function

$$J(\theta_{11}^{l=1}, \theta_{12}^{l=1}, \dots, \theta_{11}^{l=L}, \dots) = \frac{1}{n} \sum_{i=1}^n -y^{(i)} \left(\log(f_{\theta}(x^{(i)})) \right) - (1-y^{(i)}) \left(\log(1-f_{\theta}(x^{(i)})) \right)$$

where

$f_{\theta}(x_1, \dots, x_K) = a^{l=L} =$ function that computes the network's output
(output of the last neuron, in the L -th layer)

is our hypothesis

We wish to find values for each weight θ that minimize such a cost function, J

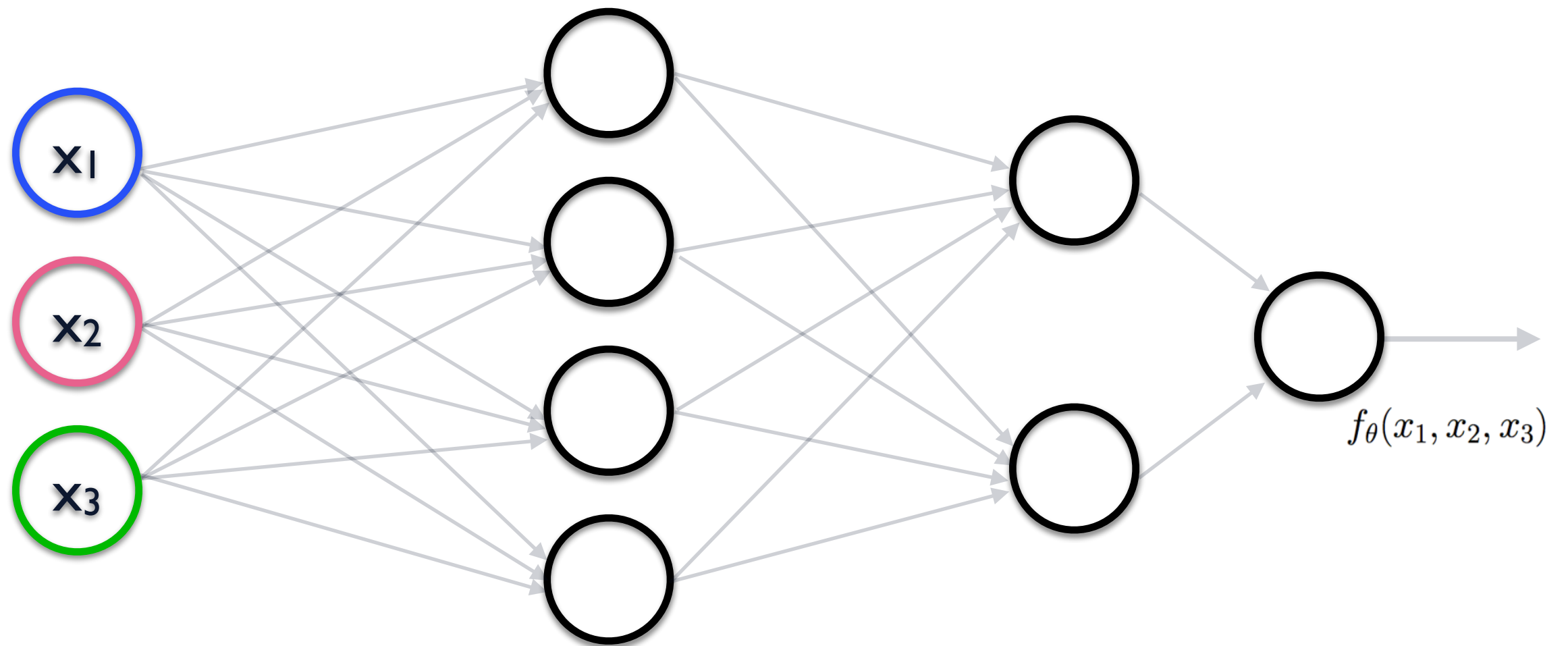
Gradient Descent Algorithm

Backpropagation Algorithm

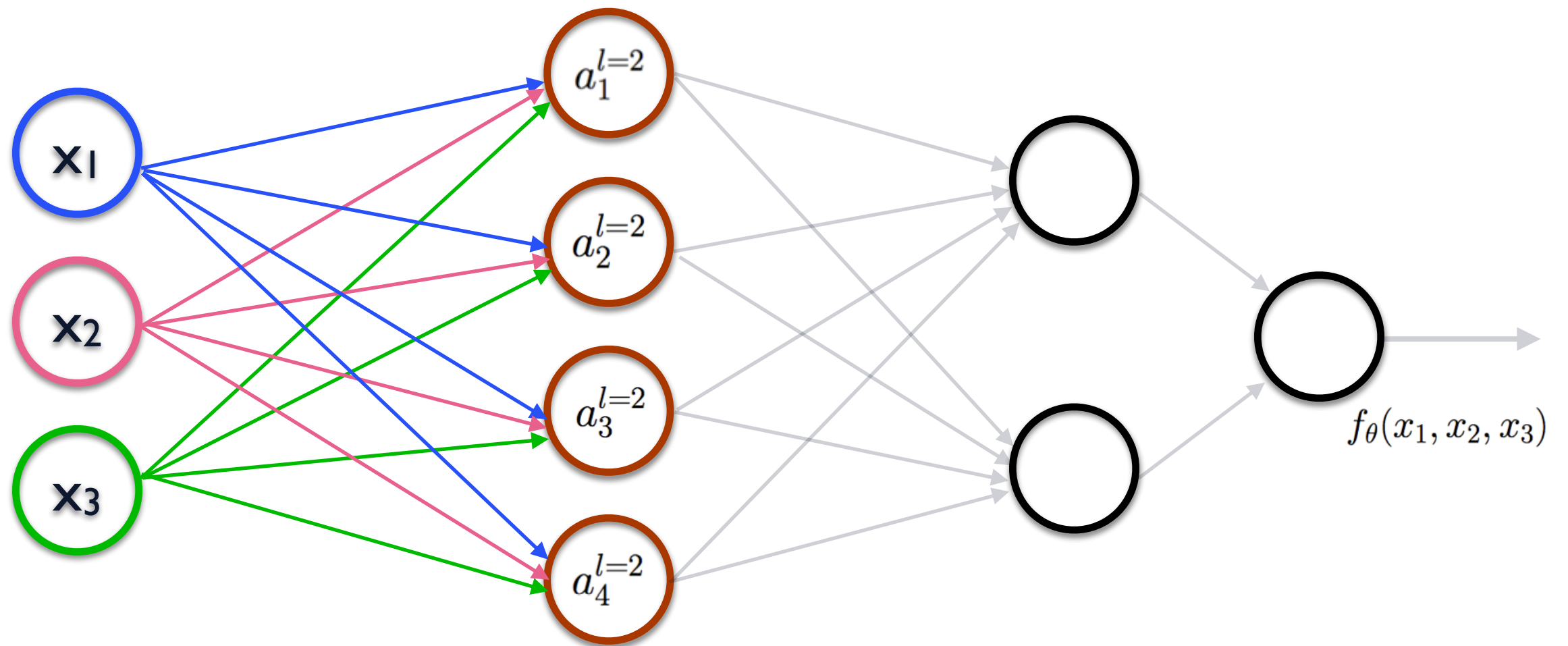
Backpropagation Algorithm

- **Training a neural network is a two-step process**
- **Forward propagation phase/step**
 - an input (instance) is **propagated** through the network, **from the first to the last layer**
 - **this produces the output/prediction made by the neural network**
- **Backpropagation (learning) phase/step**
 - this is the step where **the weights of the network are adjusted/improved**
 - **"information"** (i.e., quantities being computed) **flows from the output layer to the input layer**
 - first, we compute the difference between the network's outputs and expected outputs
 - used to assign credit (or "blame") to each neuron for the errors made by the network
 - based on this, adjusts/improves the weights of the network so it makes fewer mistakes

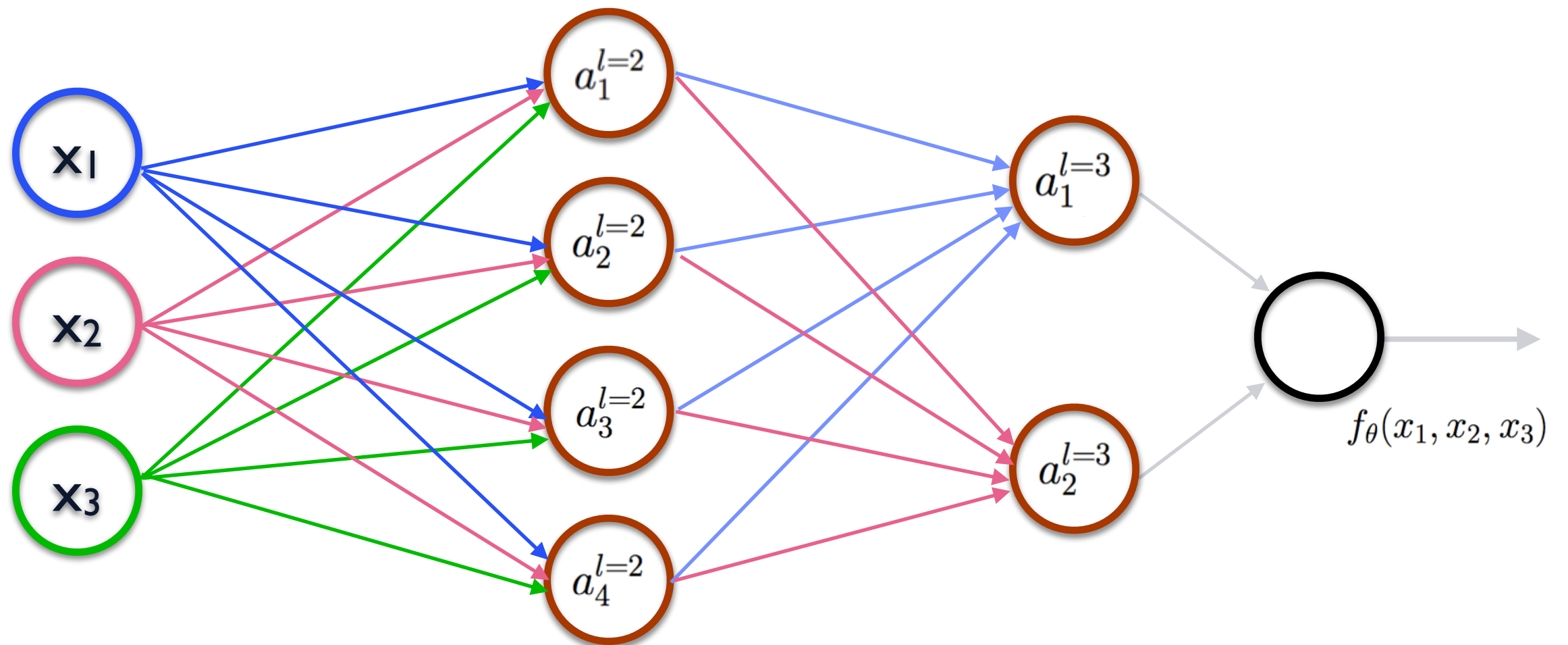
Forward Propagation



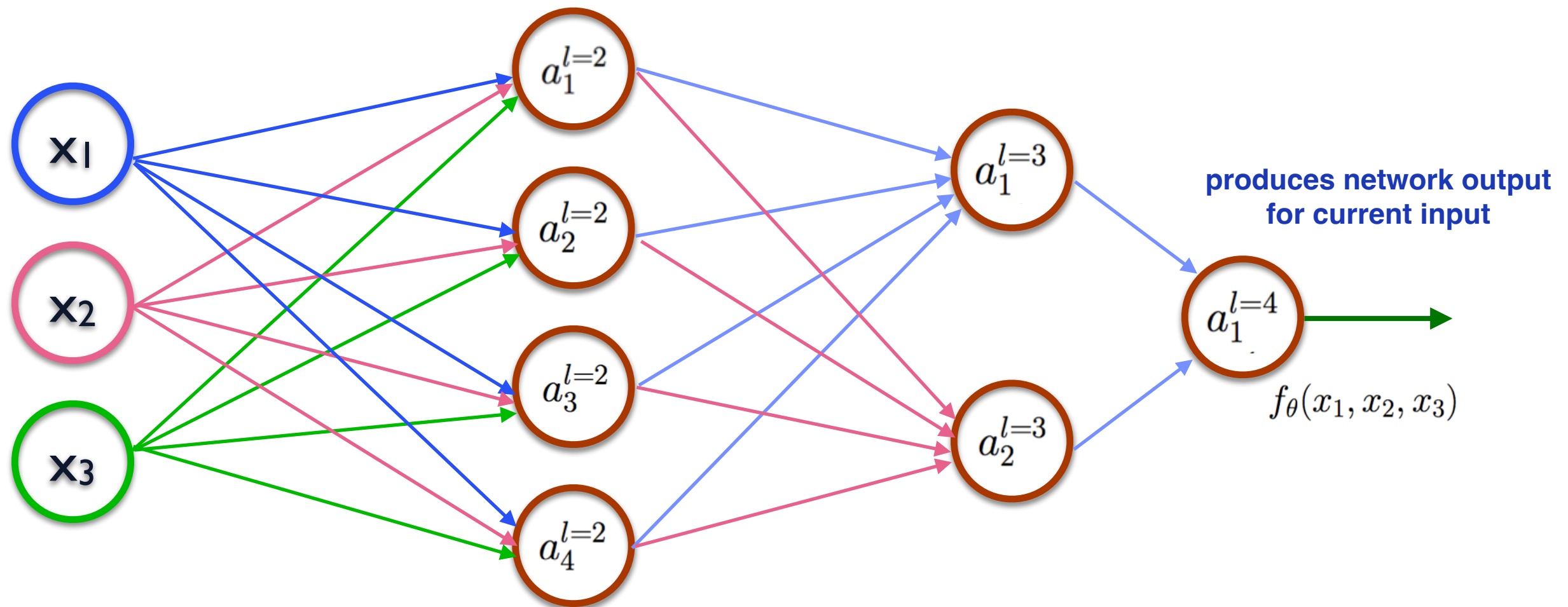
Forward Propagation



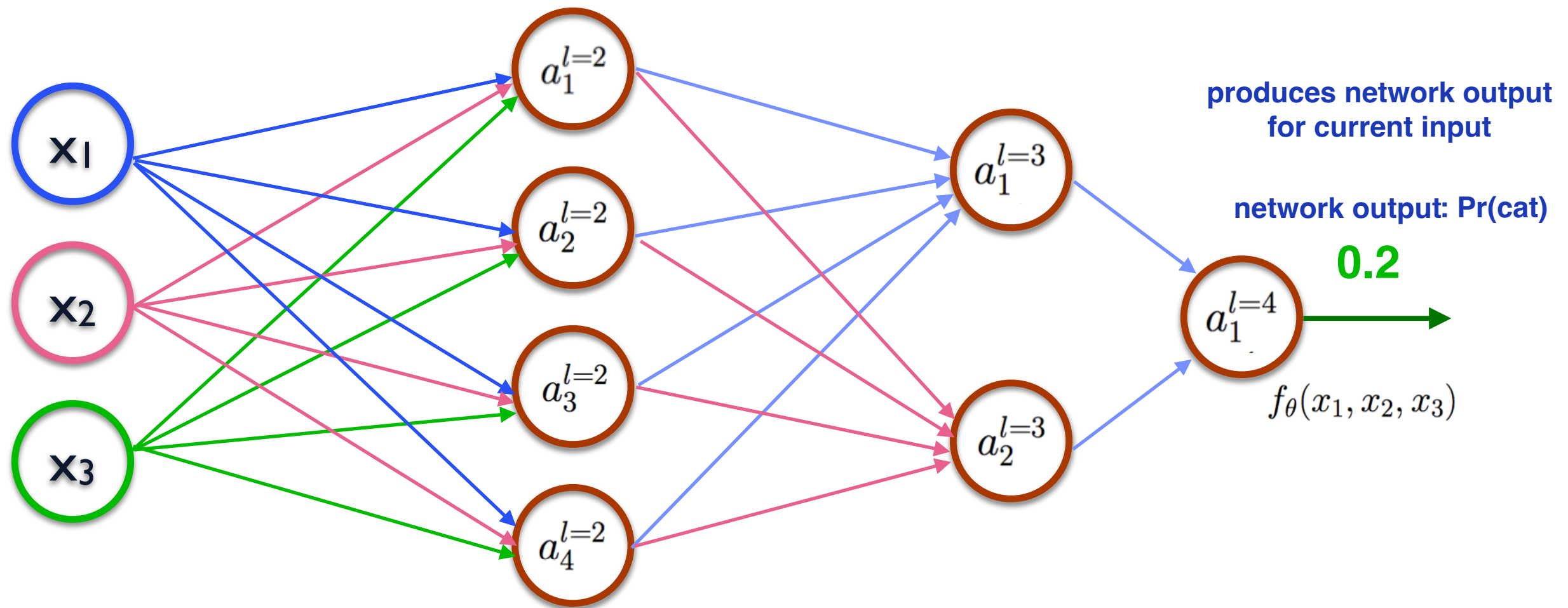
Forward Propagation



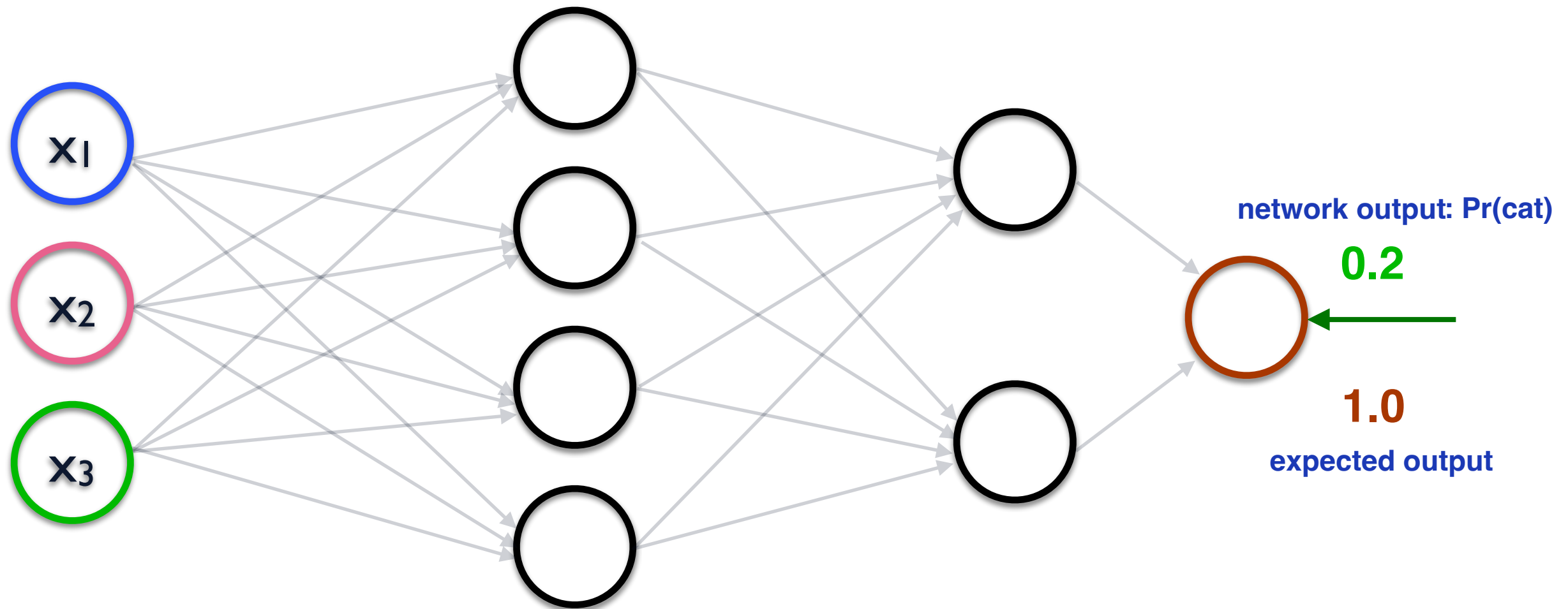
Forward Propagation



Forward Propagation



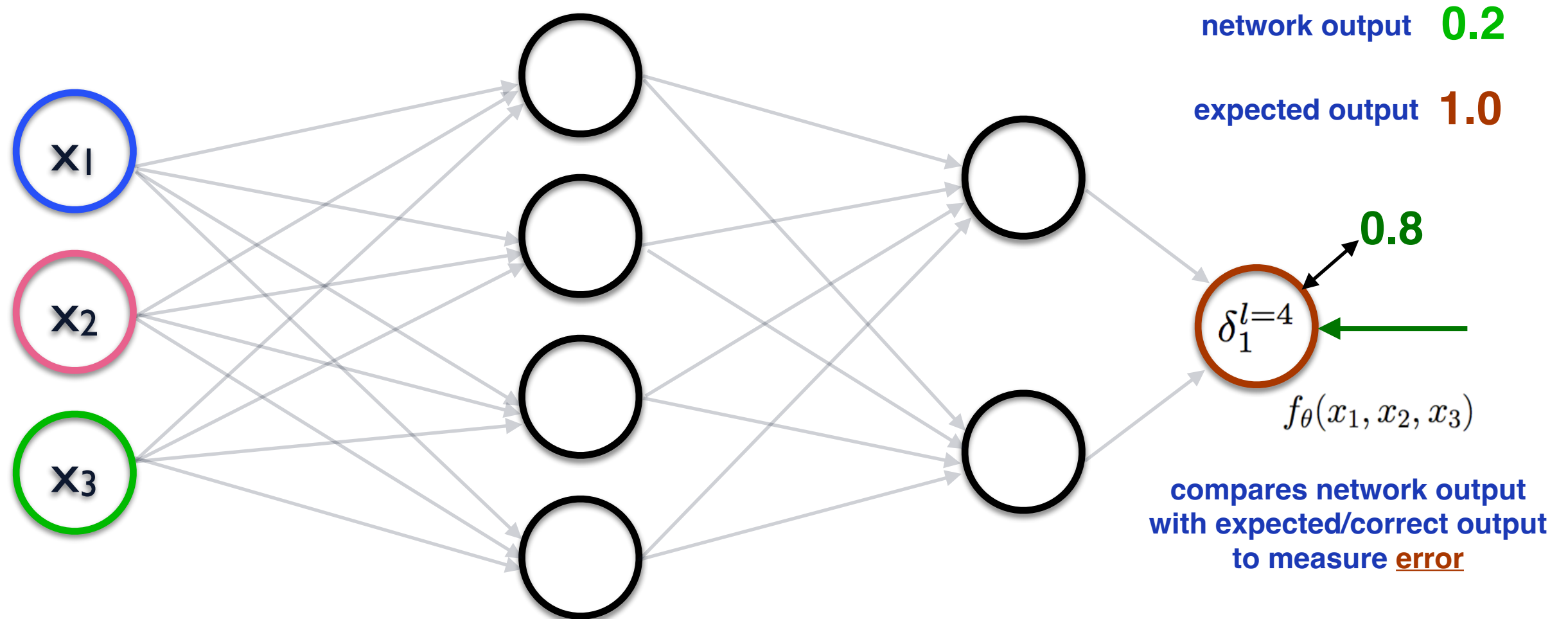
Backpropagation of Errors



compares network output
with expected/correct output
to measure error



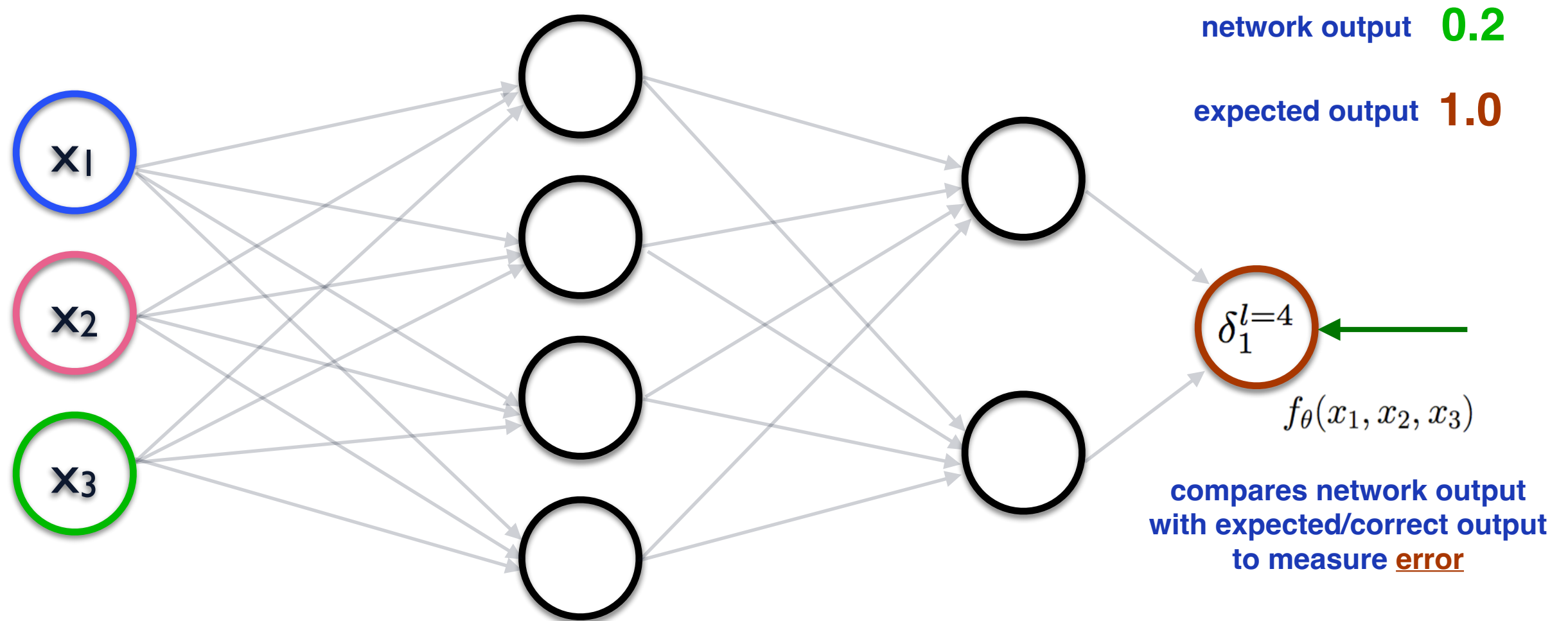
Backpropagation of Errors



Computes a δ value (one per neuron)
which will later be used to
update weights arriving at that neuron

Measures how much that neuron's output
contributed to the network error

Backpropagation of Errors



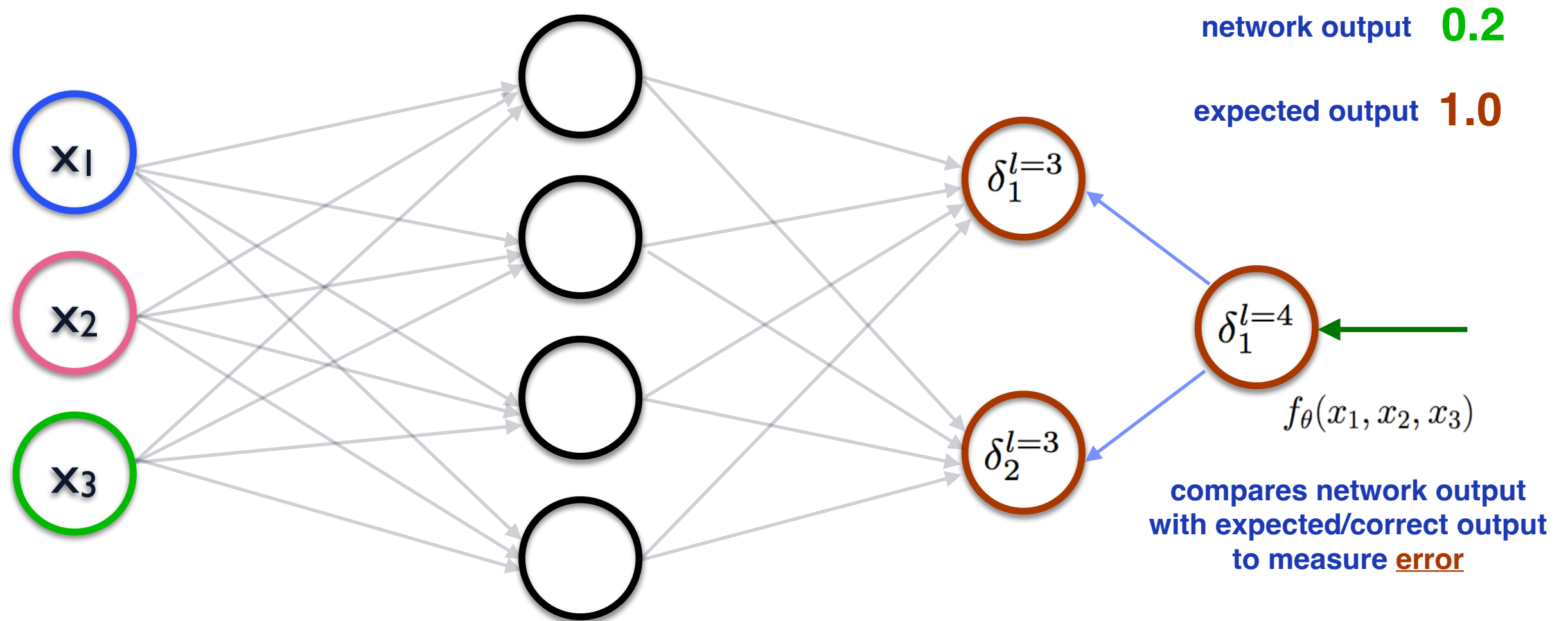
Computes a δ value (one per neuron)
which will later be used to
update weights arriving at that neuron

Measures how much that neuron's output
contributed to the network error

δ 's are computed
from the output layer to the input layer

Each δ is computed based
on δ 's of the next layer

Backpropagation of Errors



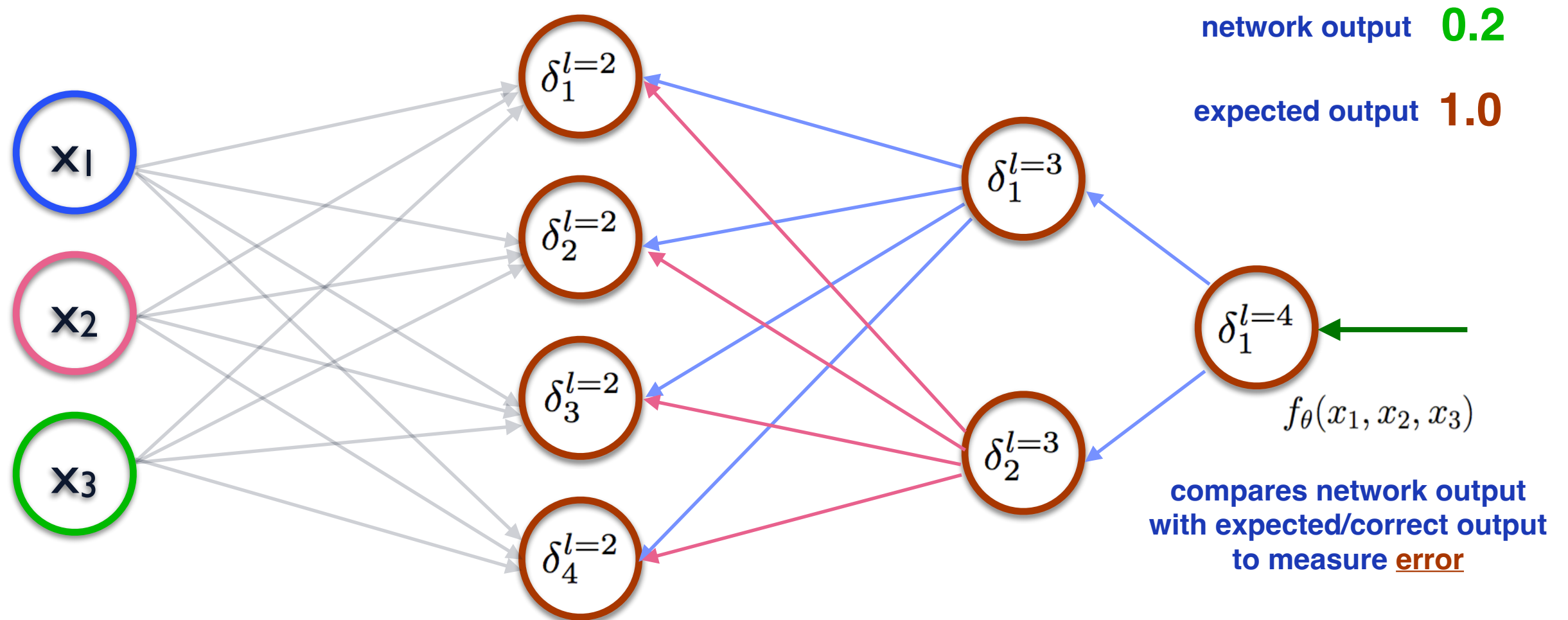
Computes a δ value (one per neuron)
which will later be used to
update weights arriving at that neuron

Measures how much that neuron's output
contributed to the network error

δ 's are computed
from the output layer to the input layer

Each δ is computed based
on δ 's of the next layer

Backpropagation of Errors



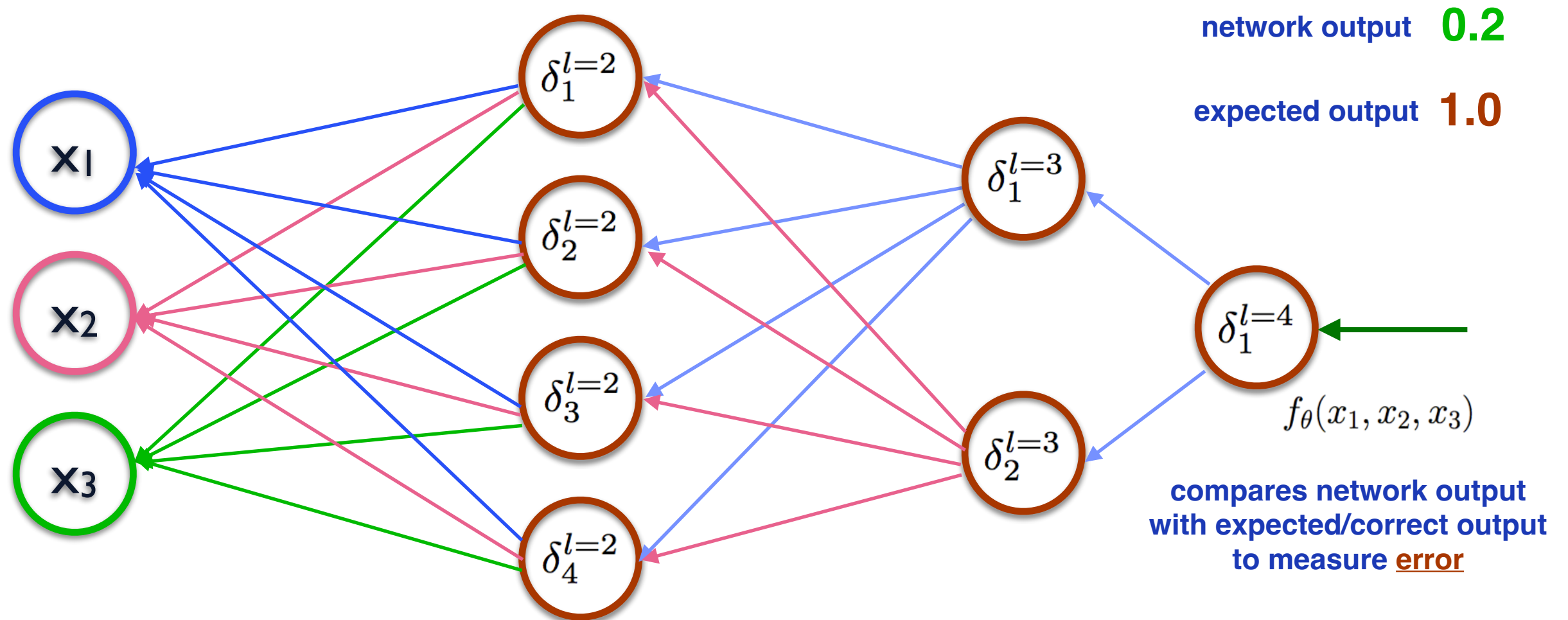
Computes a δ value (one per neuron)
which will later be used to
update weights arriving at that neuron

Measures how much that neuron's output
contributed to the network error

δ 's are computed
from the output layer to the input layer

Each δ is computed based
on δ 's of the next layer

Backpropagation of Errors



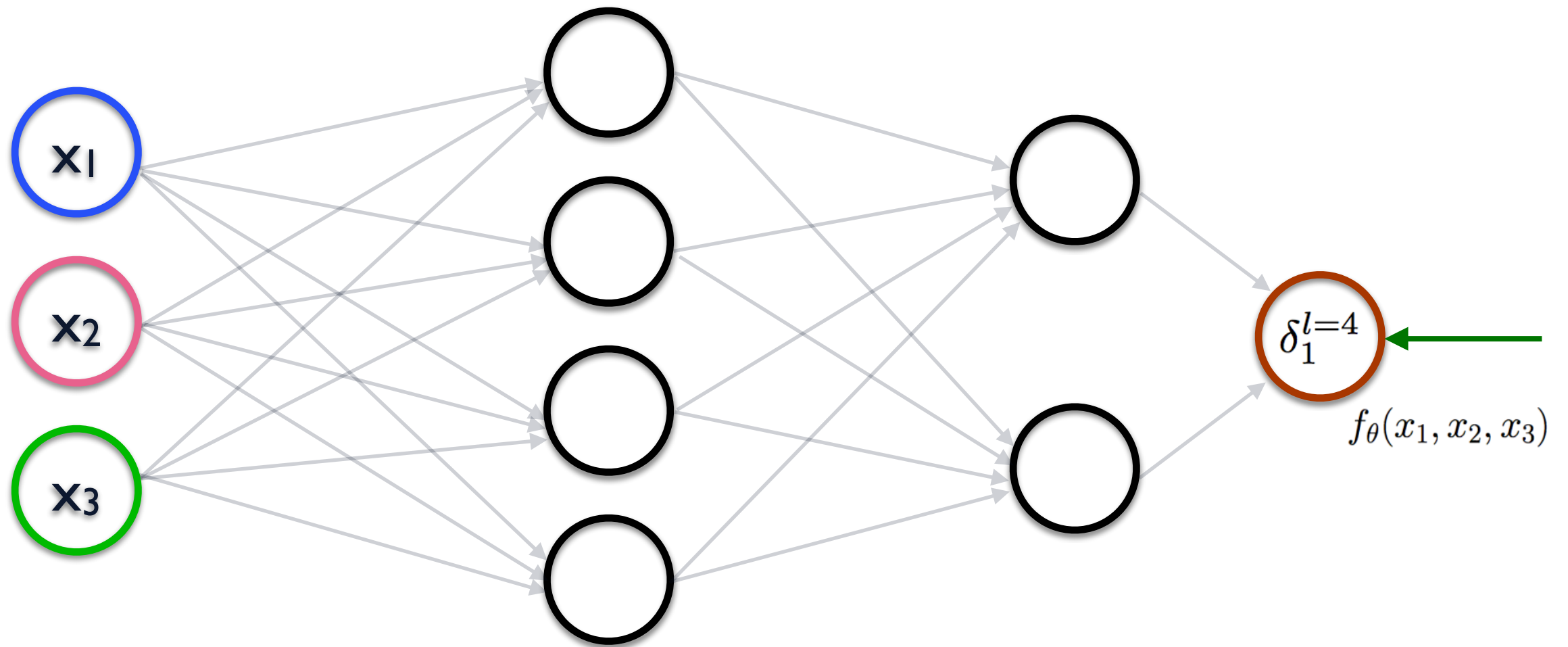
Computes a δ value (one per neuron)
which will later be used to
update weights arriving at that neuron

Measures how much that neuron's output
contributed to the network error

δ 's are computed
from the output layer to the input layer

Each δ is computed based
on δ 's of the next layer

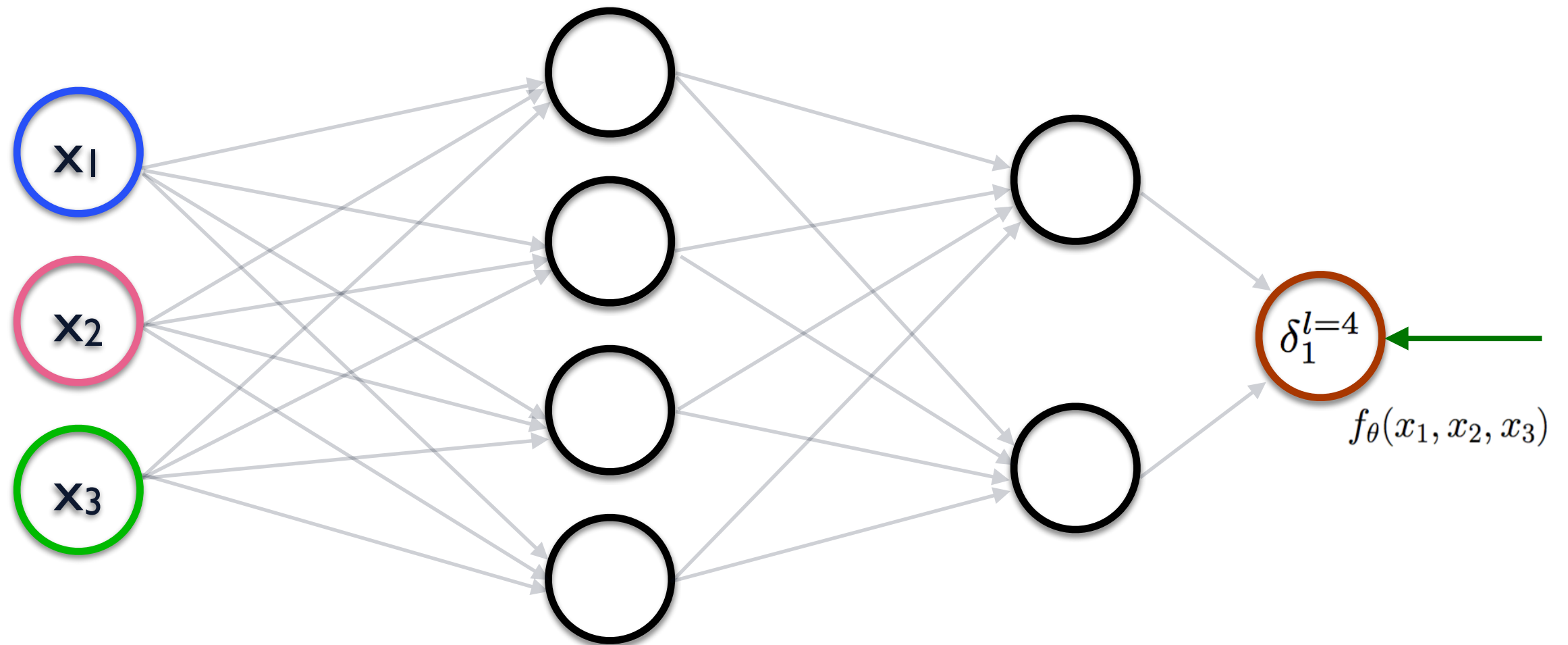
Backpropagation of Errors



δ 's tell us how to update the weights to improve the network

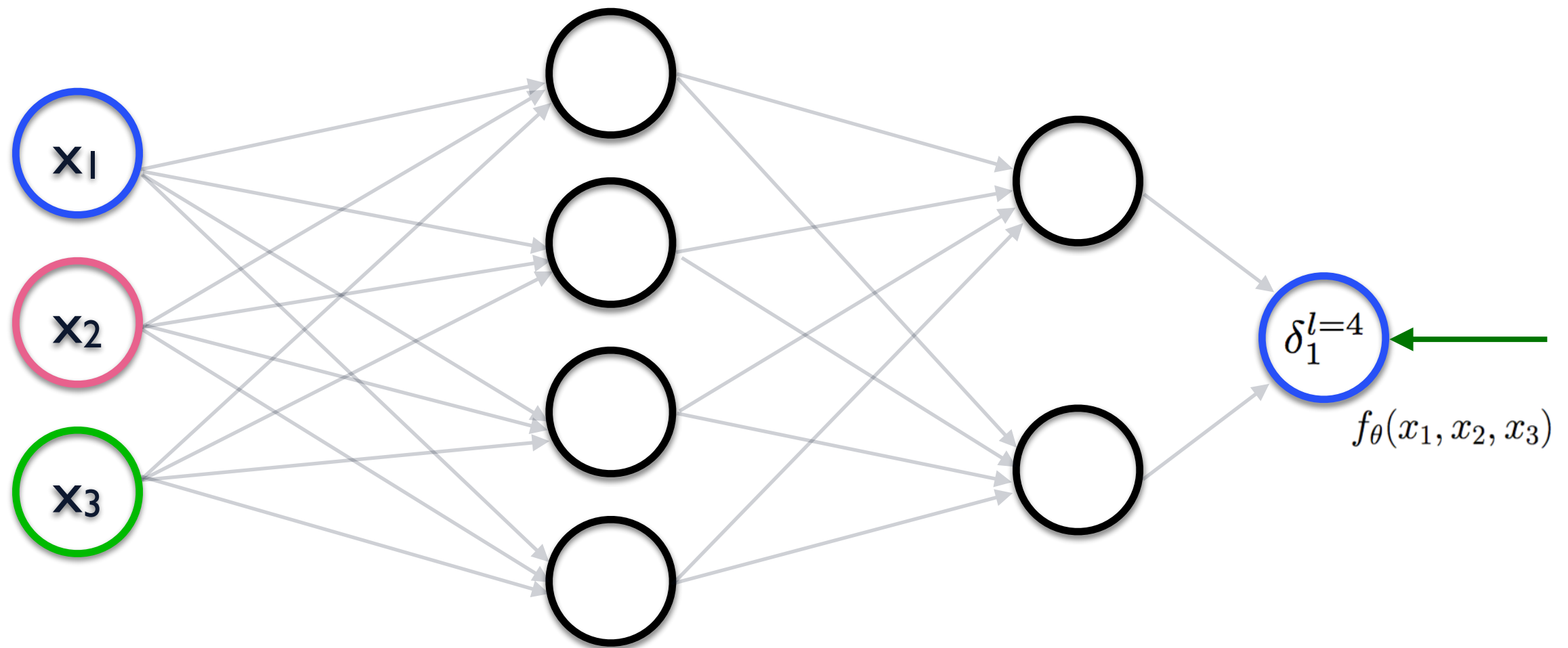
Backpropagation Algorithm

Backpropagation of Errors



How to compute the δ value of each neuron?

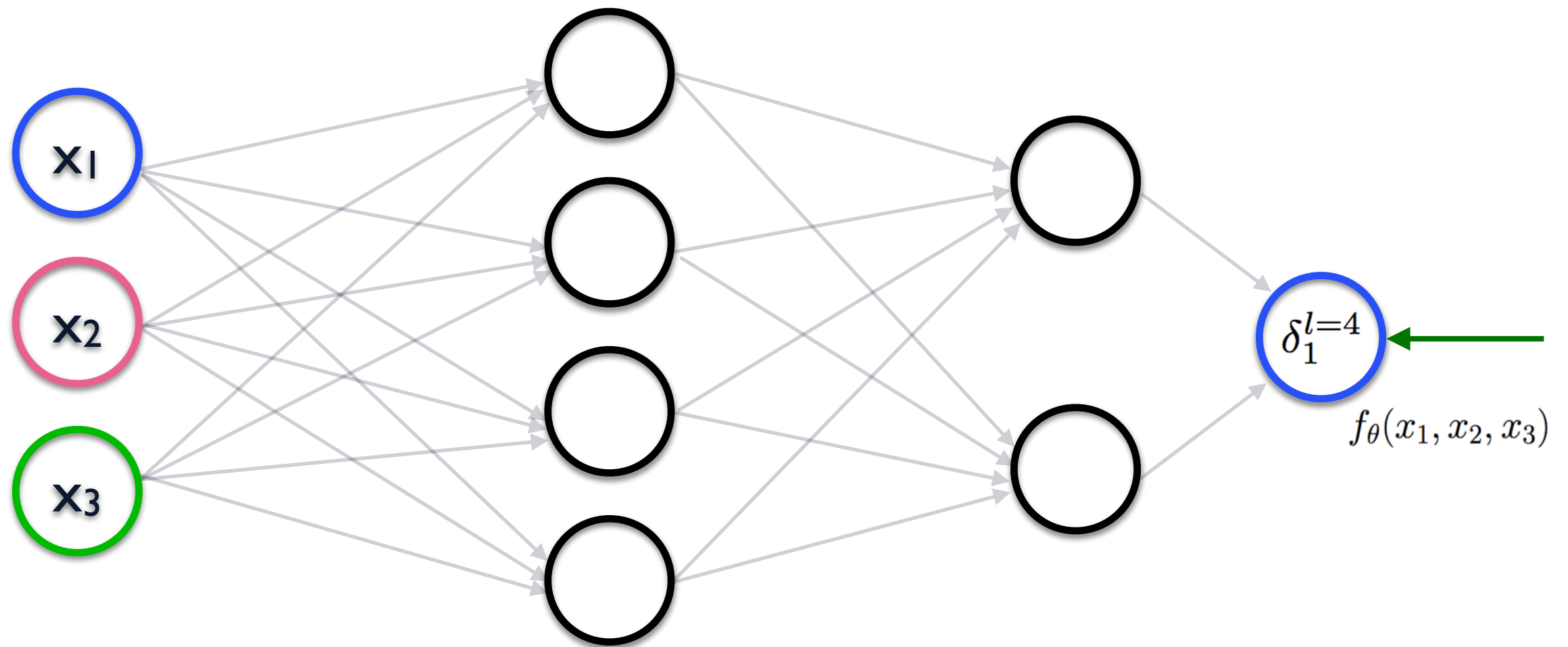
Backpropagation of Errors



$$\delta_1^{l=4} = \left(f_\theta(x_1, x_2, x_3) - y \right)$$

δ at the output layer is simply the network's prediction error

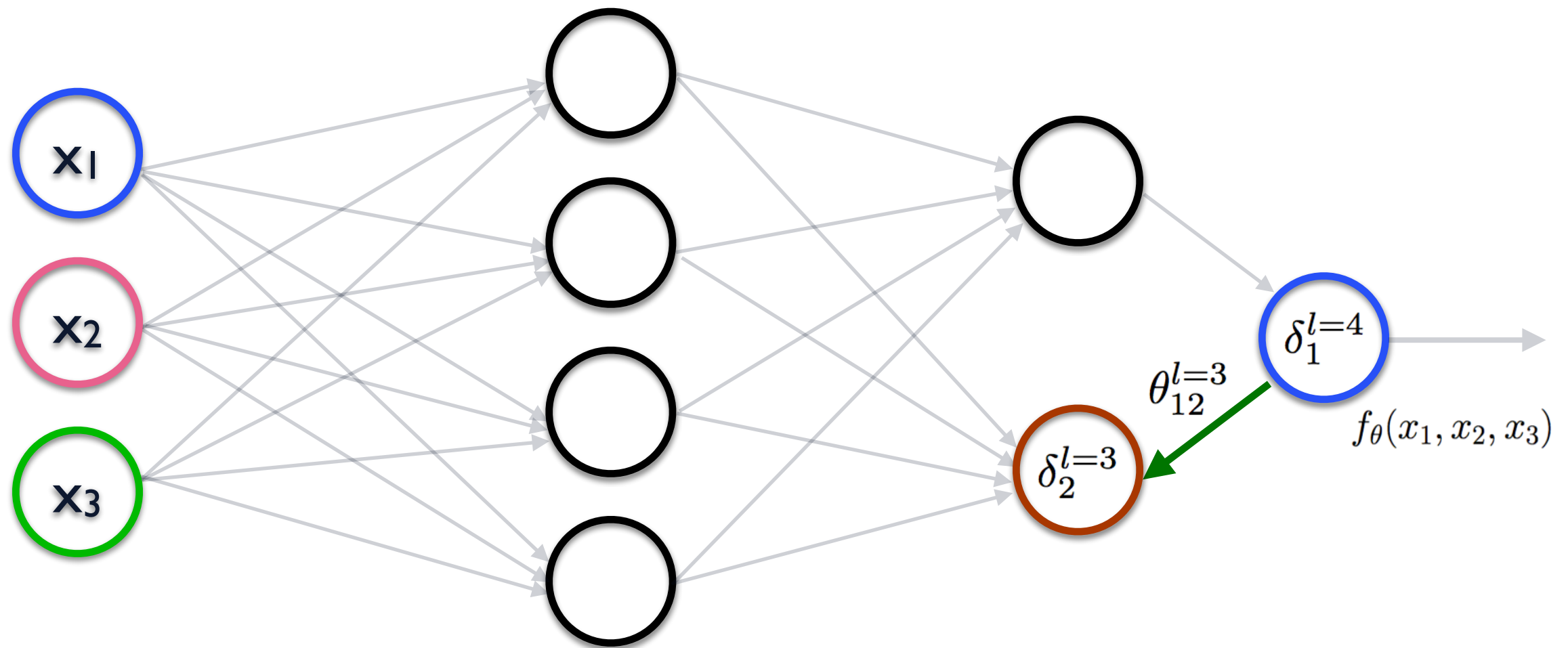
Backpropagation of Errors



$$\delta_1^{l=L} = \left(f_\theta(x) - y \right)$$

δ at the output layer is simply the network's prediction error

Backpropagation of Errors

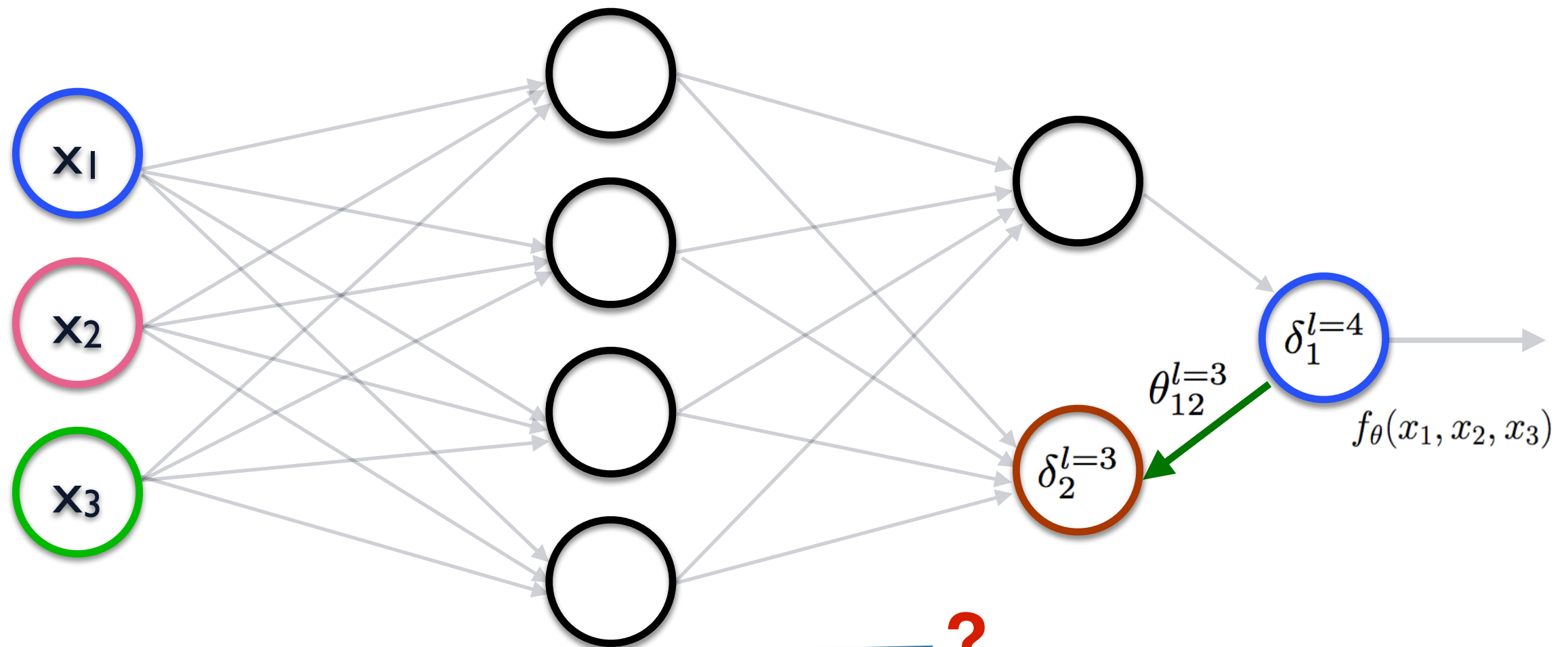


$$\delta_2^{l=3} = \left(\theta_{12}^{l=3} \delta_1^{l=4} \right) \left(a_2^{l=3} \right) \left(1 - a_2^{l=3} \right)$$

δ values of a neuron in a hidden layer:

weighted sum of the δ values of neurons in the next layer,
multiplied by the neuron's activation

Backpropagation of Errors

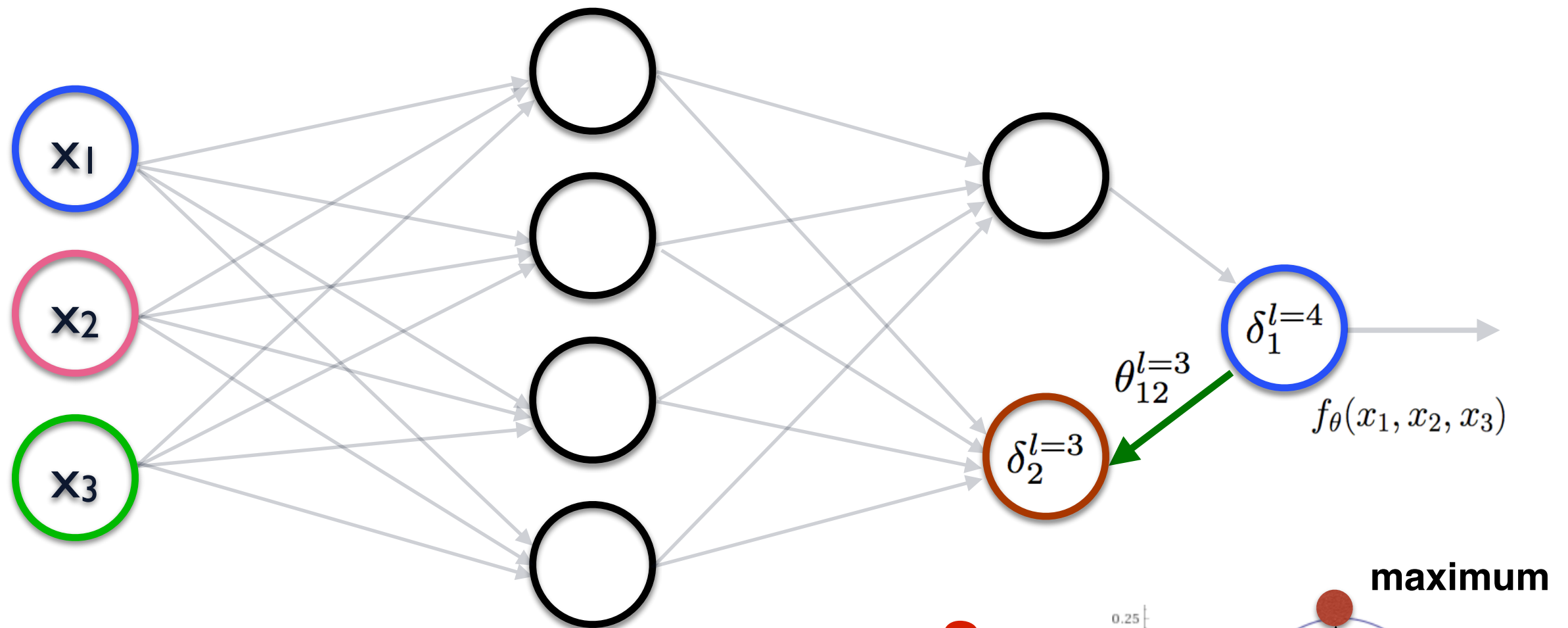


$$\delta_2^{l=3} = \left(\theta_{12}^{l=3} \delta_1^{l=4} \right) \left(a_2^{l=3} \right) \left(1 - a_2^{l=3} \right)$$

δ values of a neuron in a hidden layer:

weighted sum of the δ values of neurons in the next layer,
multiplied by the neuron's activation

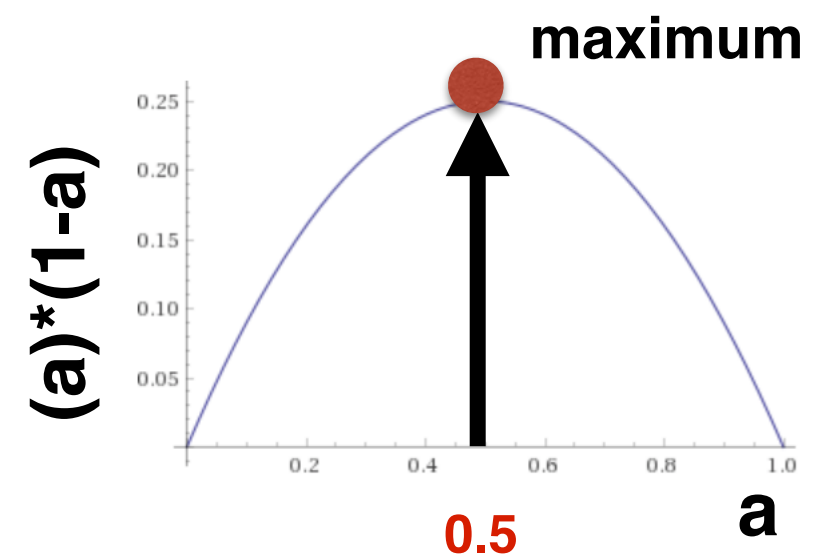
Backpropagation of Errors



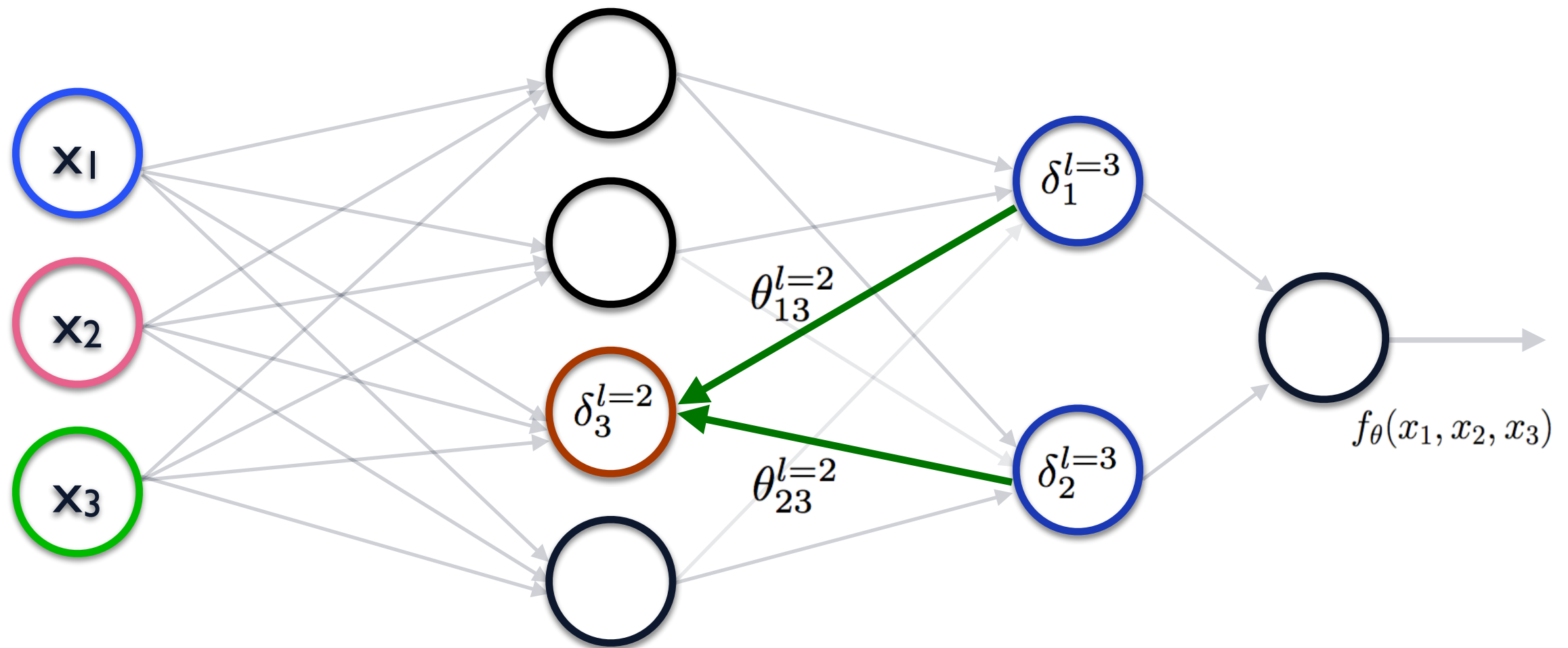
$$\delta_2^{l=3} = \left(\theta_{12}^{l=3} \delta_1^{l=4} \right) \left(a_2^{l=3} \right) \left(1 - a_2^{l=3} \right)$$

δ values of a neuron in a hidden layer:

weighted sum of the δ values of neurons in the next layer,
multiplied by the neuron's activation



Backpropagation of Errors

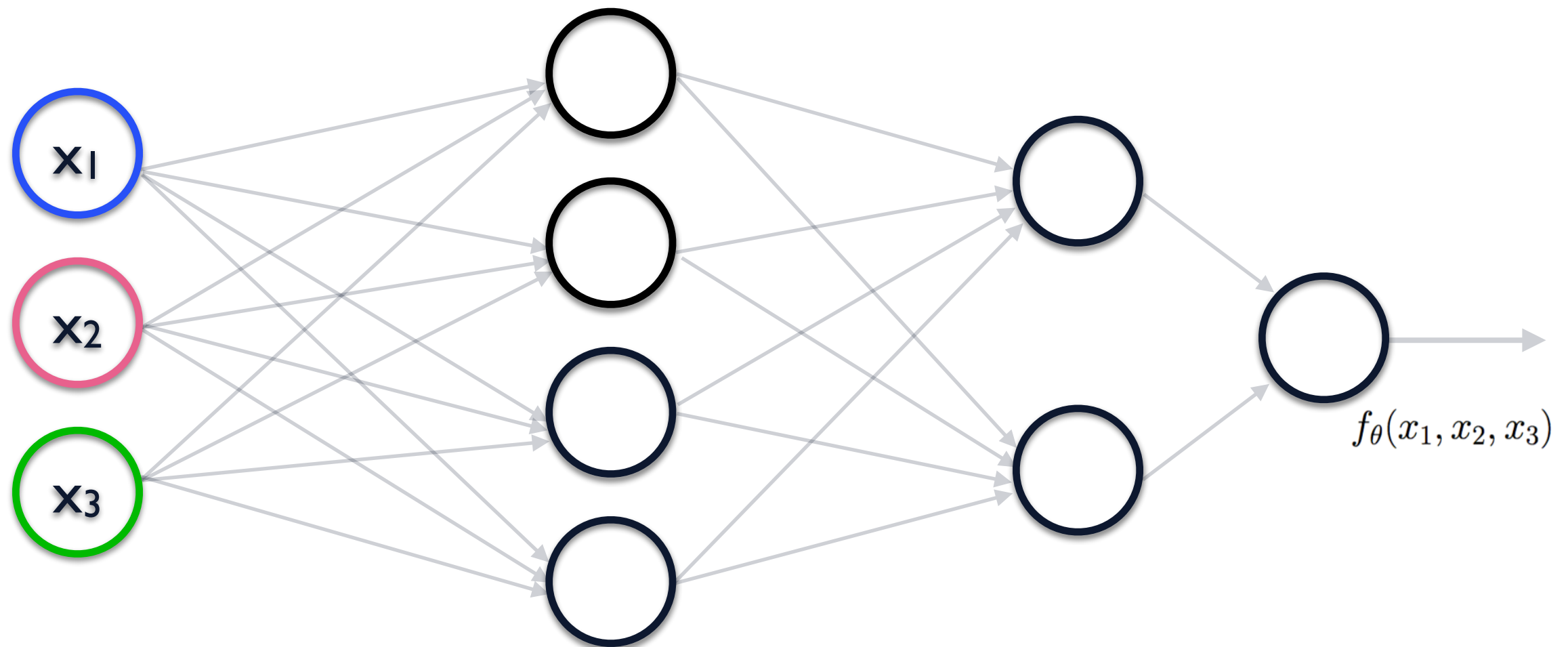


$$\delta_3^{l=2} = \left(\theta_{13}^{l=2} \delta_1^{l=3} + \theta_{23}^{l=2} \delta_2^{l=3} \right) \left(a_3^{l=2} \right) \left(1 - a_3^{l=2} \right)$$

δ values of a neuron in a hidden layer:

weighted sum of the δ values of neurons in the next layer,
multiplied by the neuron's activation

Backpropagation of Errors



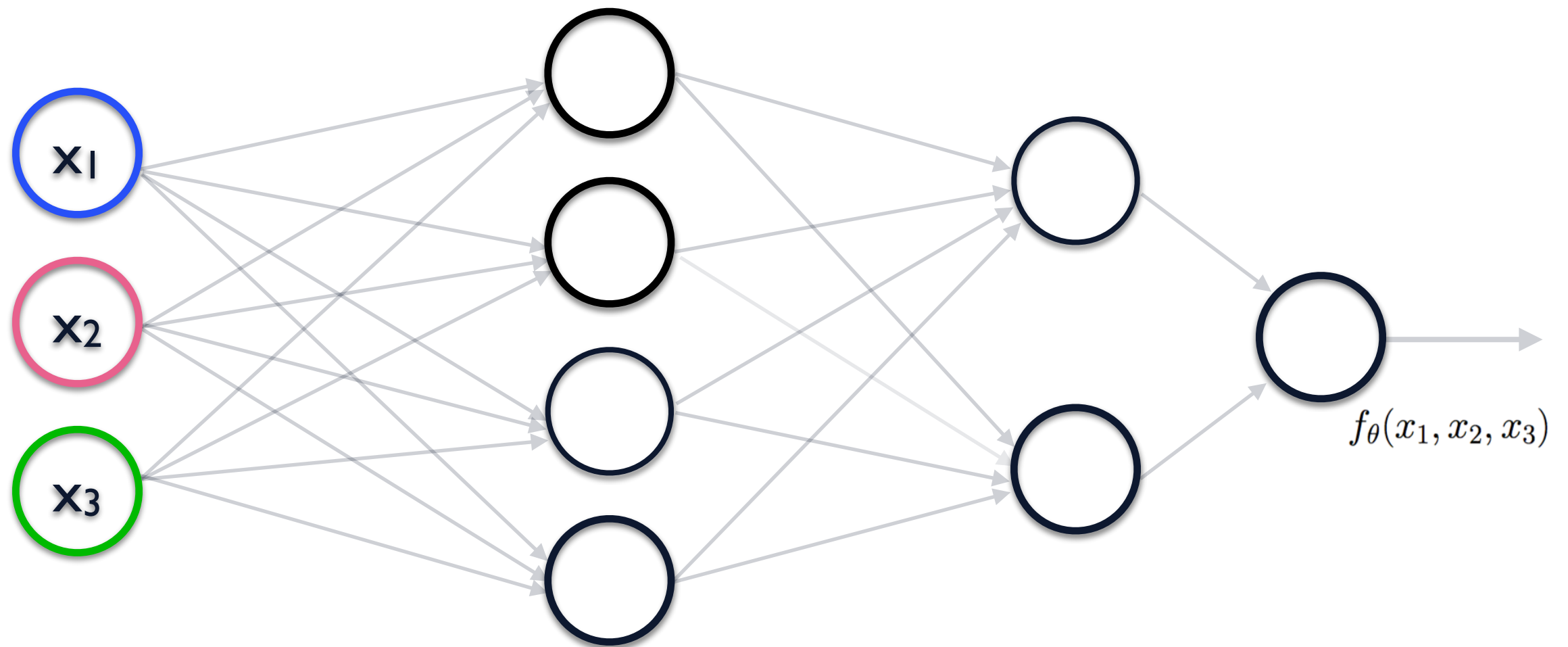
$$\delta_i^{l=k} = \left(\sum_{j=1}^N \theta_{ij}^{l=k} \delta_j^{l=k+1} \right) (a_i^{l=k}) (1 - a_i^{l=k})$$

(where N is the number of neurons in the next layer)

δ values of a neuron in a hidden layer:

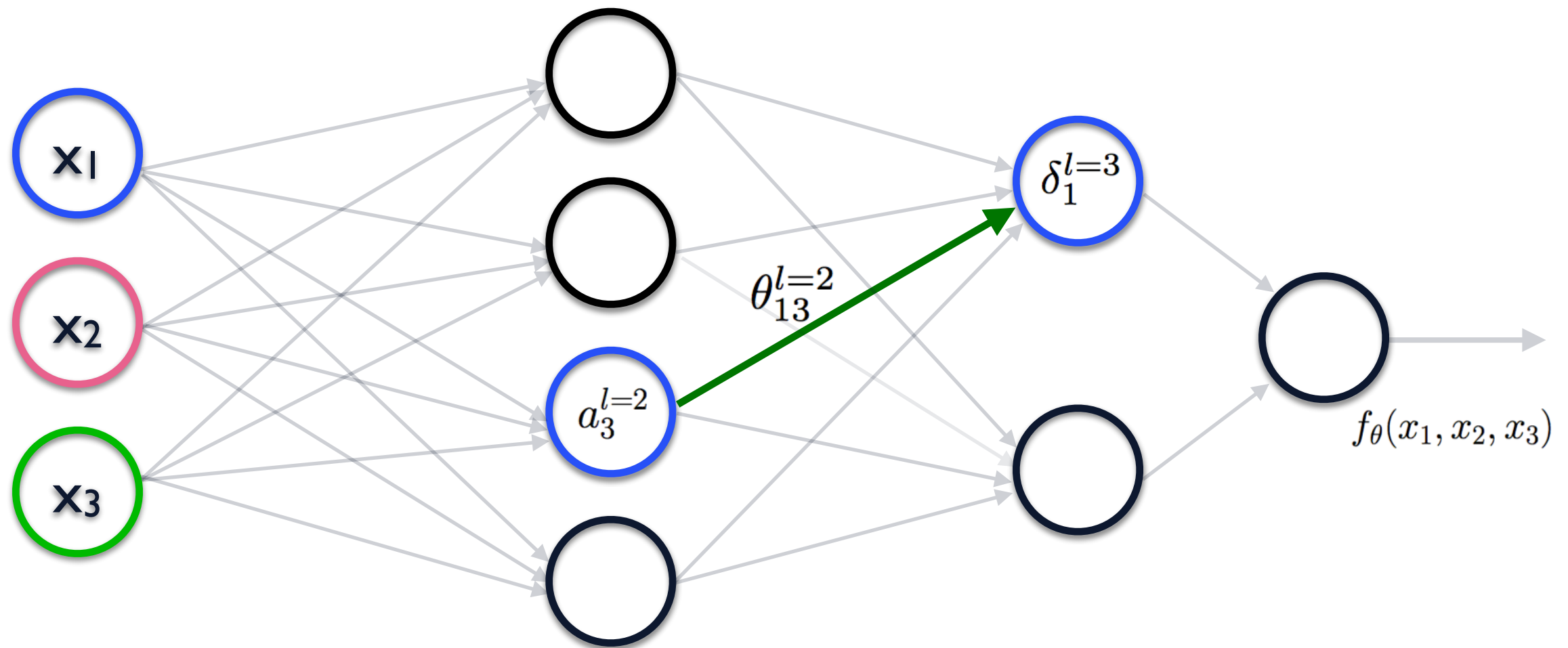
weighted sum of the δ values of neurons in the next layer,
multiplied by the neuron's activation

Backpropagation - Computing Gradients



**How to compute the gradient of each weight
based on the δ values?**

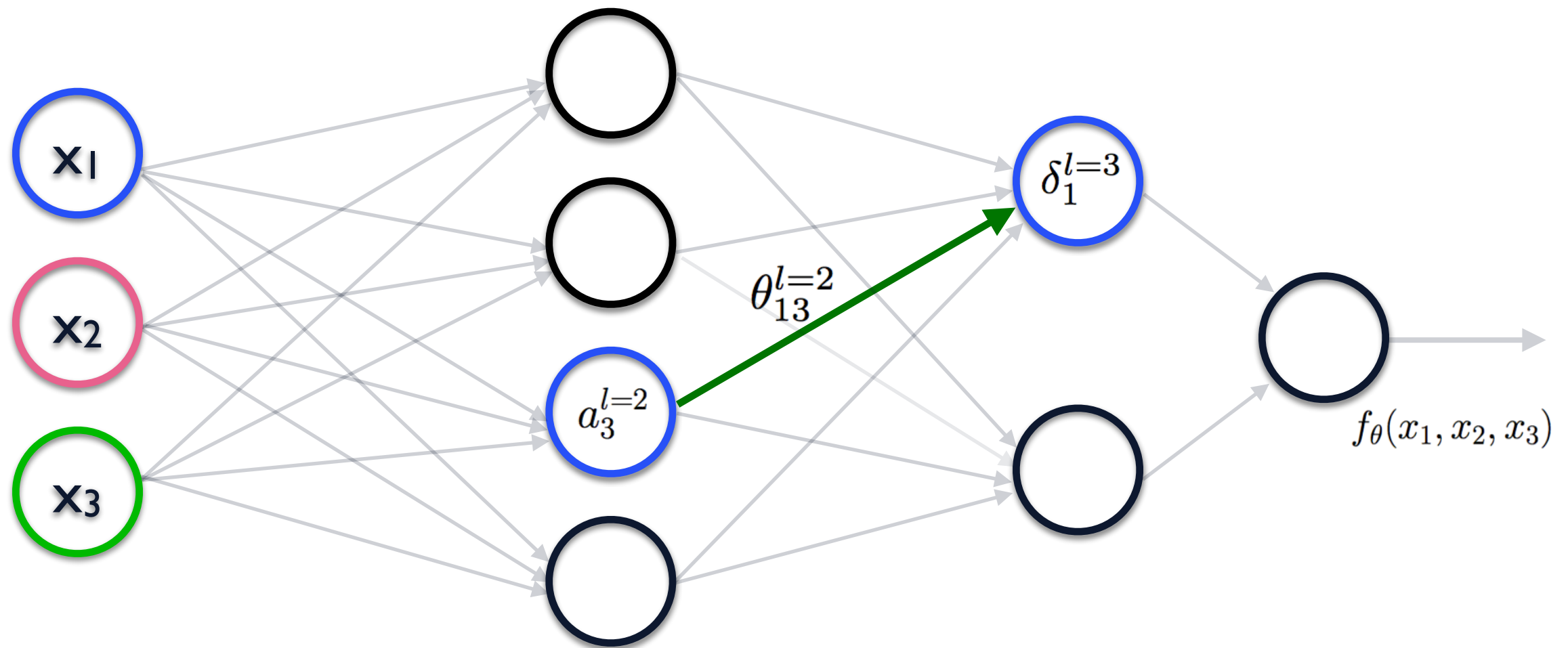
Backpropagation - Computing Gradients



$$\frac{\partial J}{\partial \theta_{13}^{l=2}} = a_3^{l=2} \delta_1^{l=3}$$

Computing the gradient of a weight based on
the error made by the network
after processing one instance/input

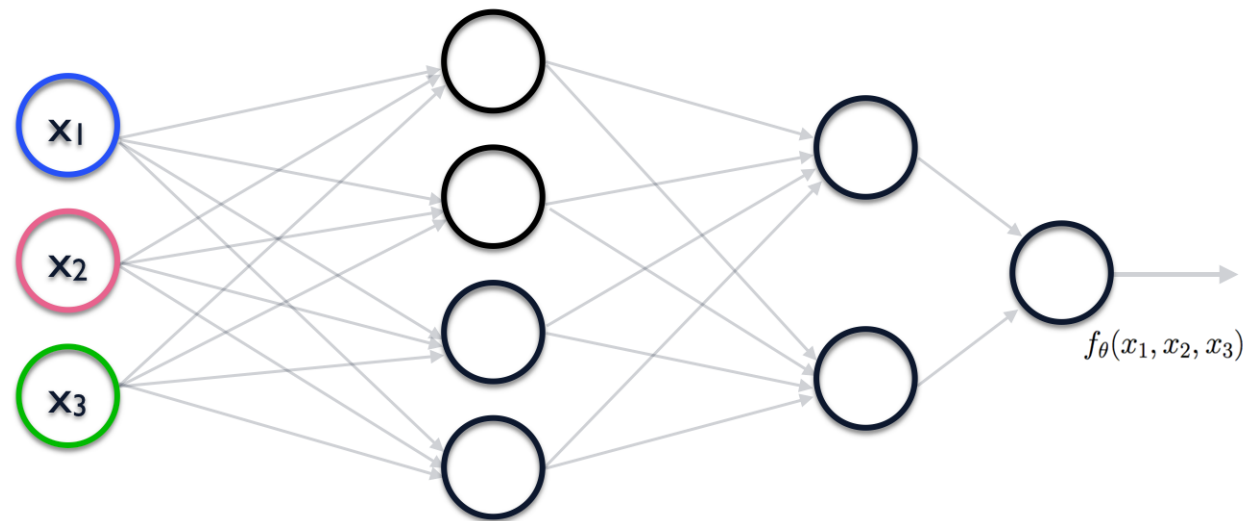
Backpropagation - Computing Gradients



$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \delta_i^{l=k+1}$$

Computing the gradient of a weight based on
the error made by the network
after processing one instance/input

Backpropagation



δ values for
neurons in the
output layer

$$\delta_1^{l=L} = (f_{\theta}(x) - y)$$

δ values for
neurons in
hidden layers

$$\delta_i^{l=k} = \left(\sum_{j=1}^N \theta_{ij}^{l=k} \delta_j^{l=k+1} \right) (a_i^{l=k}) (1 - a_i^{l=k})$$

(where N is the number of neurons
in the next layer)

gradient of
each weight

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \delta_i^{l=k+1}$$

Backpropagation - Updating Weights

We wish to find values for each weight θ that minimize such a cost function, J

Gradient Descent Algorithm

$$\delta_1^{l=L} = (f_{\theta}(x) - y)$$

$$\delta_i^{l=k} = \left(\sum_{j=1}^N \theta_{ij}^{l=k} \delta_j^{l=k+1} \right) (a_i^{l=k}) (1 - a_i^{l=k})$$

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \delta_i^{l=k+1}$$

$$\frac{\partial J}{\partial \theta_{11}^{l=1}}$$

$$\theta_{11}^{l=1} := \theta_{11}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=1}}$$

$$\frac{\partial J}{\partial \theta_{12}^{l=1}}$$

$$\theta_{12}^{l=1} := \theta_{12}^{l=1} - \alpha \frac{\partial J}{\partial \theta_{12}^{l=1}}$$

$$\vdots$$

$$\vdots$$

$$\frac{\partial J}{\partial \theta_{11}^{l=2}}$$

$$\theta_{11}^{l=2} := \theta_{11}^{l=2} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=2}}$$

$$\vdots$$

$$\vdots$$

$$\frac{\partial J}{\partial \theta_{11}^{l=L}}$$

$$\theta_{11}^{l=L} := \theta_{11}^{l=L} - \alpha \frac{\partial J}{\partial \theta_{11}^{l=L}}$$

Backpropagation Algorithm

1. Initialize all weights of the network with non-zero random values
2. For each instance $(\mathbf{x}, \mathbf{y})$ in the training set:
 1. Propagate the instance through the network, computing its output/prediction $\mathbf{f}_\theta(\mathbf{x})$
 2. Compute the δ values for all neurons in the output layer

$$\delta_1^{l=L} = (f_\theta(x) - y)$$

3. Compute the δ values for all neurons in the hidden layers:

$$\delta_i^{l=k} = \left(\sum_{j=1}^N \theta_{ij}^{l=k} \delta_j^{l=k+1} \right) (a_i^{l=k}) (1 - a_i^{l=k})$$

4. Computes the gradients of all weights of the network:

$$\frac{\partial J}{\partial \theta_{ij}^{l=k}} = a_j^{l=k} \delta_i^{l=k+1}$$

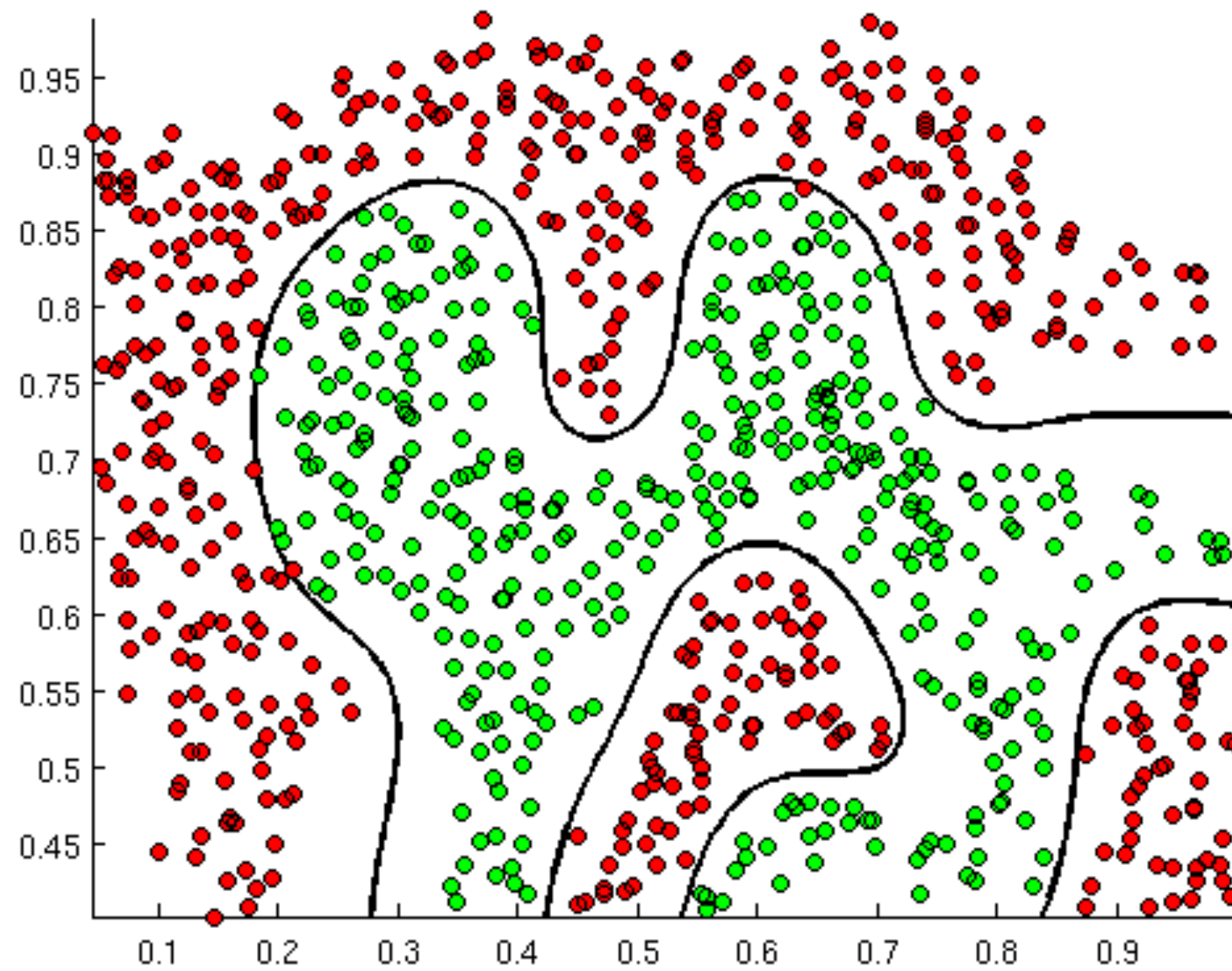
5. Adjust all weights of the network according to the gradient descent update rule:

$$\theta_{ij}^l := \theta_{ij}^l - \alpha \frac{\partial J}{\partial \theta_{ij}^l}$$

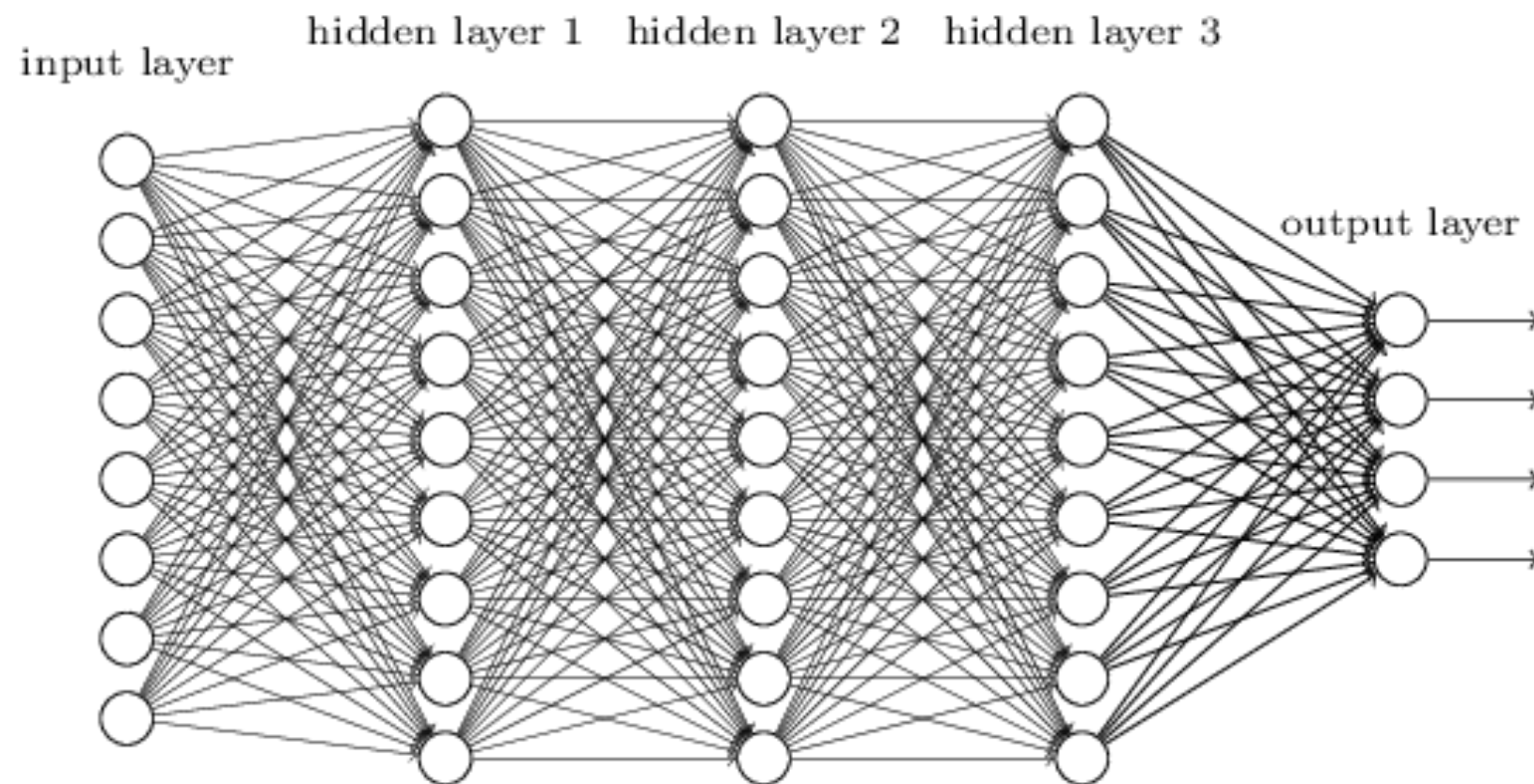
3. Evaluate the performance, $\mathbf{J}$, of the network

1. If sufficiently good (or if improvement is too small), then stop

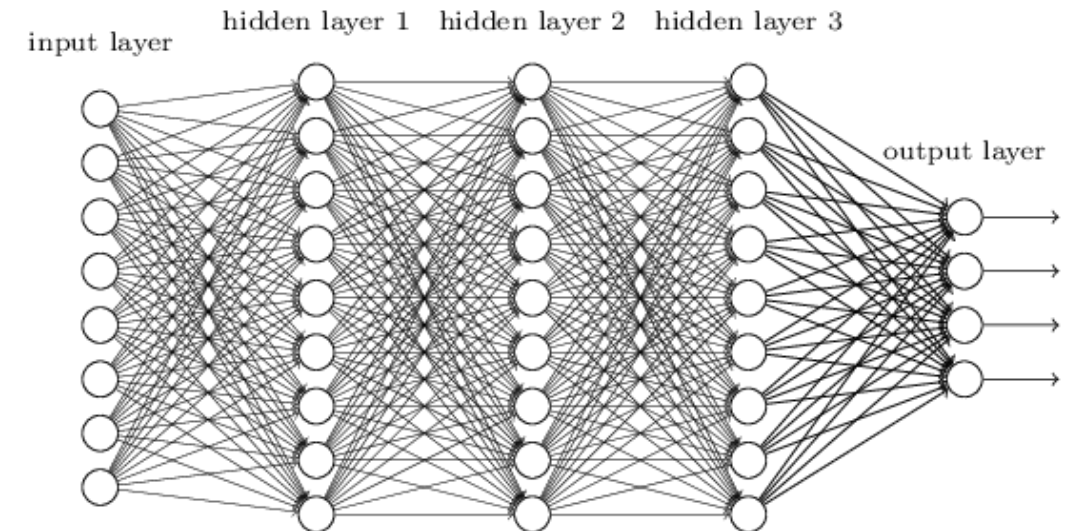
Backpropagation - Decision Boundaries



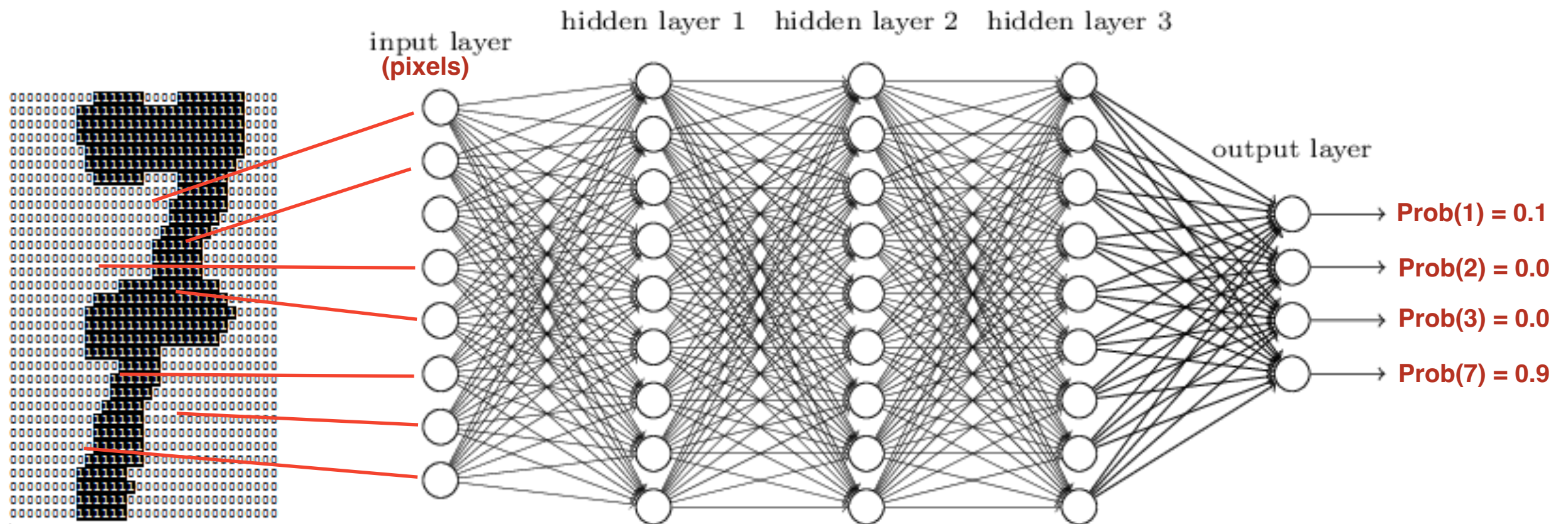
Handwriting Recognition



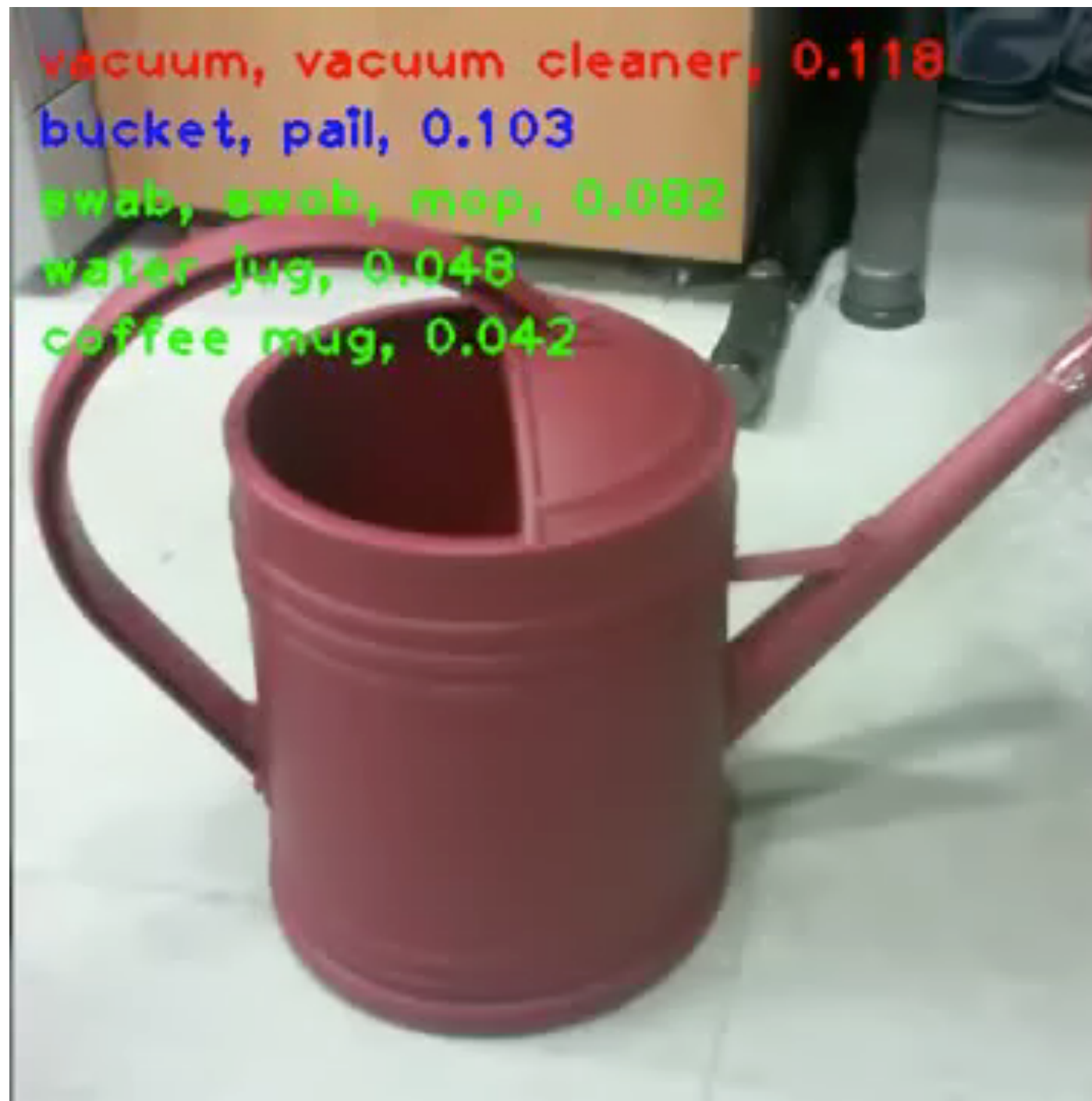
Handwriting Recognition



Handwriting Recognition



Real-Time Identification/Classification of Objects



https://www.youtube.com/watch?v=n5uP_LP9SmM

Backpropagation

Demo

playground.tensorflow.org



Machine Learning

CMPSCI 589

Bruno C. da Silva

bsilva@cs.umass.edu

Introduction to Neural Networks

UMassAmherst
College of Information
& Computer Sciences



This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/)

